

國立臺灣師範大學科技與工程學院電機工程學系

碩士論文

Department of Electrical Engineering

College of Technology and Engineering

National Taiwan Normal University

Master's Thesis

基於大型語言模型任務專屬調校：

針對算術任務設計之多答案解題系統設計

Task-Specific Fine-Tuning Based on Large Language Models:
Design of a Multi-Answer Solving System for Arithmetic Tasks

李昌育

Chang-Yu Lee

指導教授：賴以威 博士

Advisor: I-Wei Lai Ph.D.

中華民國 113 年 7 月

July, 2024

致謝

在 2018 年，我進入師大電機系就讀，並在大學一年級遇到了教學工數課的賴以威老師，同時也是我的大學專題指導教授與碩士論文指導教授。感謝老師讓我自由選擇論文研究方向，並且透過討論培養了論文閱讀、問題分析、解決問題、上台報告等專業能力。在研究期間也給予了我很多在外授課的機會，提升了在台上講課的經驗，讓我對在眾人面前發表想法以及專業內容時毫不退卻。與以威老師交談的期間，讓我能夠更能培養對研究或者是生活上的批判與欣賞的眼光，不僅在學術上更在人生道理上，以威老師是我這 6 年來的導師。

感謝峻逸，有想法以及行動力的你，一直是我在研究道路上的好夥伴，儘管我們兩人研究內容有巨大的差異，但都會時常討論許多內容，讓我受益良多。感謝巧心，在研究所期間帶給我歡樂，讓我了解更多不同的思考方式。感謝學長宗勝、暉家以及學姊梓沂給予許多有建設性的建議，讓我學會如何與他人共同開發專案，除了學術上在生涯規畫上也有很大的幫助。謝謝學長俊賢能夠認真以及直接地給出建議。

感謝我的家人們，時常的關心我以及給予我精神上的支持，以及時常掛念著我。感謝我的同學們-青霖、宇達、宣錚，這過去 6 年中有你們的陪伴相當的快樂，儘管有時有些瘋狂的想法，但因如此才讓我大開眼界。

基於大型語言模型任務專屬調校：針對算術任務設計之多答案解題系統設計

學生：李昌育

指導教授：賴以威 博士

國立臺灣師範大學電機工程學系碩士班

摘要

隨著大型語言模型(Large Language Model, LLM)的出現，線上教育等內容也逐漸轉型，開始使用 LLM 作為平台的主要核心以減少在金錢以及時間上的花費，儘管目前有許多學科已逐漸使用 LLM 作為知識來源，但在數學教育上的發展較為緩慢，因為數學算術問題的多變性以及解題的多樣性。在過往的研究中提出在數學學習中若給予學生多種不同的解題方向，則會提升學習者對於數學的理解以及學習的熱情。

本篇研究提出運用於 LLM 的微調設計方法。在過往的研究中數學算術任務在 NLP 中一直是難以解決的領域，目前有許多研究致力於提升 LLM 的計算能力透過固定化計算策略，然而這樣的方式難以運用至教育領域中。本篇研究中，我們提出三種微調資料集設計之方法只需透過監督式微調技術，使得 LLM 能夠在一次的推理中針對數學算術問題生成多種解題方向。此微調資料集設計方法能夠在本任務中達到約 77\%的平均正確率。此外，本篇論文亦討論三種設計方式對多答案解系統之影響。

關鍵字：大型語言模型、數學、教育、微調

Task-Specific Fine-Tuning Based on Large Language Models: Design of a Multi-Answer Solving System for Arithmetic Tasks

Student: Chang-Yu Lee

Advisors: Dr. I-Wei Lai

Department of Electrical Engineering
National Taiwan Normal University

ABSTRACT

With the emergence of Large Language Models (LLMs), online education and other content are gradually transitioning, beginning to use LLMs as the primary core of the platform to reduce costs in terms of both money and time. Although many disciplines have gradually adopted LLMs as a source of knowledge, the development in mathematical education has been slow due to the variability of its problems and the diversity of solutions. Previous studies have suggested that providing students with various problem-solving approaches in mathematical learning enhances learners' understanding of mathematics and enthusiasm for learning.

This study proposes a fine-tuning design method applied to LLMs. In previous research, mathematical arithmetic tasks have always been a challenging area in Natural Language Processing (NLP). Currently, many studies are dedicated to enhancing the computational capabilities of LLMs through fixed calculation strategies; however, such methods are difficult to apply in the educational field. In this study, we propose three fine-tuning dataset design methods that only require supervised fine-tuning techniques, enabling LLMs to generate multiple problem-solving approaches for mathematical arithmetic problems in a single inference. This fine-tuning dataset design method achieves an average accuracy of approximately 77% in this task. Furthermore, this paper discusses the impact of the three design methods on multi-answer solution systems.

Keywords: LLMs, Mathematics, Education, Fine-tune

目錄

致謝	i
目錄	iv
圖目錄	vi
表目錄	vii
第一章 介紹	1
1.1 研究動機	1
1.2 相關文獻	2
1.3 主要貢獻	3
1.4 論文架構	3
第二章 背景知識	4
2.1 數位學習	4
2.2 大型語言模型	6
2.2.1 大型語言模型概論	6
2.2.2 語言模型基礎架構	8
2.2.3 微調概述	13
2.2.4 在教育上的應用	15
2.2.5 在算術任務上的困難	18
2.3 針對算術任務微調的大型語言模型	19

第三章	微調設計	23
3.1	斷詞方式探討	24
3.2	數據集設計	25
3.2.1	數學運算代理	26
3.2.2	答案生成的問題關聯控制	28
3.2.3	答案生成的序列控制	30
第四章	實驗結果與分析	33
4.1	正確率	33
4.1.1	斷詞系統效能對比	34
4.1.2	資料集設計效能對比	35
4.2	多樣性	45
4.2.1	獨特性	45
4.2.2	豐富性	45
4.3	消融實驗 Ablation study	47
4.4	錯誤分析	50
第五章	結論與未來研究	52
附錄 A	— Dataset	54
A.1	Train dataset	54
A.2	Test dataset	57
參考文獻		66
自傳與學術成就		68

圖目錄

2.1	Chat-GPT 針對算術問題生成之解答多樣性	6
2.2	大型語言模型基礎架構圖	8
2.3	範例斷詞後的結果	8
2.4	Word embedding 範例	9
2.5	Transformer 架構圖	10
2.6	大型語言模型推理流程	13
2.7	LoRA 概念圖，其中輸入的維度為 d ，輸出維度 z ，以及 $r \ll d \wedge r \ll z$	15
2.8	Wolfram alpha 範例	17
2.9	GOAT 在乘法問題的處理方式	21
2.10	GOAT 在除法問題的處理方式	22
3.1	本研究之系統架構	24
4.1	兩種斷詞方法的平均正確率	36
4.2	4 種不同資料集配置的平均正確率	38
4.3	Accuracy difference 與 Function 使用率與 Function 使用並正確之比較	40
4.4	此圖展示模型在不同配置下對前三個回答 (A、B、C) 的準確率	42
4.5	「Function+Relevance-Control」與「Function+Relevance-Control+Sequence-Control」的全部錯誤率，600 為測試題目的總數	43
4.6	此圖展示有無 Sequence-Control 在不同數值數量下全部錯誤的機率	44
4.7	不同模型對於在不同數值數量下的平均解答數量	47
4.8	「Function+Relevance-Control」、「Function+Sequence-Control」、和「Function+Relevance-Control+Sequence-Control」平均正確率比較圖	48
4.9	此柱狀圖展示了模型在不同配置下對首三個回答 (A、B、C) 的準確率	49
4.10	在不同數值數量下，正確率與結構相似率的比較	51

表目錄

2.1	在 MathGLM 中用於訓練之範例	20
3.1	在不同斷詞系統下的斷詞結果	24
3.2	此表呈現在 GPT 以及 Llama 兩種斷詞系統下的數字的斷詞結果	25
3.3	此表呈現使用 Function 的資料集設計方式	27
3.4	此表為 CAS 系統之輸入以及輸出	27
3.5	此表呈現使用 Relevance-Control 的資料集設計方式	29
3.6	此表呈現使用 Sequence control 的資料集設計方式	32
4.1	在「Function」配置下，錯誤類型的數量統計	41
4.2	此表呈現不同 Dataset 的唯一率	45
4.3	此表呈現不同 Dataset 的平均解答數量	46
4.4	不同數值數量下的平均正確率以及平均正確解答數量	50
A.1	訓練資料集的問題符號表示。	54
A.2	測試資料集的問題符號表示	58

第一章 介紹

1.1 研究動機

隨著 2020 年新冠肺炎 (Covid-19) 的爆發，全球範圍內的居家隔離措施徹底改變了人類的生活方式。原本需要前往辦公室的上班族轉向居家工作，學生則從校園學習轉變為遠程教育。網路使用率的增加伴隨著數位學習 (E-Learning) 概念的快速普及。據統計，自疫情爆發以來，線上教育平台的使用率呈倍數增長，至今仍在持續上升，顯示線上學習已成為一種不可逆轉的趨勢。

2023 年，由 OpenAI 公司開發的大型語言模型 (LLM) Chat-GPT 憑借其強大的功能及泛用性，顛覆了傳統的線上教育平台技術構成。眾多教育平台紛紛將 LLM 作為基礎技術，無論在用戶溝通還是問題解答方面都取得顯著成果，特別是在語文教育領域取得了突出成就，甚至擴展至程式語言學習，大幅降低了學習各類知識的門檻。然而，與上述應用相比，將 LLM 運用於數學教育的案例相對較少，主要原因是 LLM 在解決數學計算問題時面臨困難。

目前許多研究致力於提升 LLM 在解決數學問題的準確率，並已取得良好成效。然而，這些研究主要集中在是否能夠獲得正確答案，而在教育領域，解題的準確率固然重要，解題過程的完整性也不容忽視。教育的目的不僅是教授計算技巧，更重要的是培養學生的思考模式。[1, 2] 指出，當學生接觸到多樣的解題思路時，他們對數學的理解會進一步加深，同時也能激發學習的興趣和動力。因此，我們的主要目標是通過微

調 LLM，使其能夠針對算術問題提出多種解題方法。

本文將從資料收集與處理到模型效能評估等方面，深入探討 LLM 在算術任務中生成多種解答的表現。

1.2 相關文獻

在數學教育中，單靠記憶和重複計算方法遠遠不夠，更重要的是學生能夠理解和把握數學概念及其之間的關係。正如 [1] 中所指出，通過實施”數字談話”這一策略，學生可以自由地提出各種解決問題的方法，這不僅提高了他們對數學的理解，還激發了他們的學習興趣。學生在這個過程中享受提出自己的策略，並對其他同學的策略表現出興趣，這種互動促進了深層次的理解和興趣的培養 [2]。

然而，如 [2] 所述，完全依賴教師來提供多樣化的計算策略不僅資源消耗大，也可能限制了教育的彈性和創新性。這突顯了尋找替代方法以提高教學效率和質量的必要性，例如利用技術工具或平台來生成多樣化的解題策略，從而減輕教師的負擔並提供更豐富的學習資源。在這個背景下，大型語言模型（LLM）的應用成為了一種潛在的解決方案，可以自動生成不同的計算策略，以支持數學教育的多樣化需求。

有研究 [3, 4] 旨在提升 LLM 在數學計算上的能力，其中透過固定化計算策略的方式降低問題的複雜度，[4] 在提升正確率的任務上取得了可觀的成果，但這樣的方式卻與教育所需要的內容相反，因為教育更重視的是學生的理解深度和解題過程的多樣性。進一步研究表明，透過引入多種解題路徑不僅可以提升學生對數學問題的總體理解，還能促進他們的創新思維和問題解決能力。

在此背景下，LLM 的進一步應用需關注不僅是問題的正確解答，更應該著重於解題過程的多樣化呈現。這樣的變化不僅能夠提供給學生多角度的思考模式，也能有效地激發他們探索未知的興趣，這對於教育的長遠發展是至關重要的。此外，LLM 的這一應用也呼應了 [1] 中提倡的教育理念，即教育應該鼓勵學生展開討論，自由地表達和

驗證自己的想法，而不是僅僅追求答案的正確性。

1.3 主要貢獻

1. 深入探討自然語言處理中的數學問題解決策略：我們的研究著重於自然語言處理（NLP）領域中一個特定的分支——數學問題解決。考慮到斷詞系統對於 NLP 任務的重大影響，本研究深入分析了不同斷詞策略對於解決數學問題的具體影響，揭示了斷詞細節對算法性能的決定性作用。
2. 資料集設計策略以增強大型語言模型的多重解答能力：我們開發了一套專為算術問題設計的 LLM 微調資料集，此設計特別強調生成多樣化的解答方式。透過實證研究，此方法不僅豐富了模型處理問題的策略，同時也在提升平均正確率上達到了顯著的效果。

1.4 論文架構

本篇論文將於第二章節介紹本論文討論之系統背景知識。第三章介紹針對 LLM 微調設計之細節。第四章將介紹模擬方法與結果。第五章為結論與未來研究。

第二章 背景知識

本章節旨在介紹本研究的先備知識。本文在 2.1 節會概述現存的數學學習平台；第 2.2 節則會介紹本研究的技術主軸大型語言模型；在 2.3 節會介紹到目前為止已經針對數學領域所開發的大型語言模型，包括研究目標與訓練改良方式。

2.1 數位學習

隨著數位科技的發展蓬勃，數位線上學習的教育方式已經普及到世界各地。數位學習是利用數位化的技術來提供學生可以學習的資源、進行教學、以及評估學習結果的一種學習方式。數位學習可以提供學習者不同的上課體驗，為學習者提供多元、彈性、客製化的學習體驗。

數位學習與傳統教育現場的最大的差別在於，課上的時間與位置皆沒有限制，學生可以在任何地點與時間進行教學。無論是教學者還是學習者，只要有網路連接的場所就可以進行學習。在數位學習中，學習者可以在不同的地點、不同的裝置、不固定的時間進行學習，瀏覽教材、完成作業、進行評量。相較於傳統的教育現場，學生會有更好的自主性，以及學生可以透過不同種類的教材進行學習，例如：影片、題目卷等，可以提高學生對於學習的興趣以及學習的效率。然而數位學習仍面對不少的挑戰以及困難，儘管科技的快速進步，但傳統教育方式卻仍是不可被完全取代的手段。

線上學習 (online learning) 是數位學習的一種，而線上學習又可以分為同步學習 (synchronous learning) 和非同步 (asynchronous learning) 兩種學習模式。其中同步學習

是指學生以及教師同時進行教學與學習，例如：即時會議等；非同步教學則是指學生與教師在非同一時間進行教學與學習，例如：錄製與觀看影片等。

在同步學習中，教師扮演著傳遞知識的角色，除此之外也會透過觀察學習者的反應對教學內容進行客製化，例如：提問、轉換說明方式等等。然而，在非同步教學中因為教師與學習者非即時的互動，所以學習者只能透過預先錄製好的訊息進行學習，若遇到學習上的困難只能透過留言等非即時性的方式取得回應。因為無法即時取得更進一步的解說或者是互動，容易會造成學習者的意願下降或者是學習效率低下。

在非同步學習中，學習者接觸的教材大部分皆經過教師處理以及重新編排，這樣的方式會造成教師會需要花費大量的時間在準備教材以及大量的金錢在人事支出上。又因為教材需要透過人工的方式進行編排以及審查，往往會造成學習者難以快速的接觸到最新的教材。

在非同步學習環境中，教材的即時更新通常不可能實現，因此教材主要由教師負責處理和分析解釋。當學習者遇到困難時，他們可獲得的知識來源僅限於教師提供的解答。這種模式極其適合於語文學科或非數學類學科，因為這些學科的問題解答往往有固定的模式。例如，在英語學習中，填空題可以通過「分析文法」或「字彙的含義」來選擇答案。然而，這種方法不適用於數學學科，因為數學解題具有高度的多元性。同一數學問題可能由不同的人解答，每個人的解題過程和方法都可能有所不同。圖2.1展示了數學領域中解題方法的多樣性和複雜性，強調了不同解法之間的差異和多樣性。這種多元性對教育者提出了額外的挑戰，需要他們在準備教材時考慮到各種可能的解答方式，以豐富學生的學習經驗並促進深入理解。

為了解決非同步學習的教材更新困難，有許多的線上學習平台紛紛導入了大型語言模型作為教材開發的幫手。



You

幫我用兩種不同的方法計算 89×15



ChatGPT

計算 89×15 的兩種方法得到的結果都是 1335：

1. 直接乘法：使用普通的乘法運算， $89 \times 15 = 1335$ 。
2. 加法模擬：將 89 加 15 次，即重複加 89 一共 15 次，最終結果也是 1335。[-]

圖 2.1: Chat-GPT 針對算術問題生成之解答多樣性

2.2 大型語言模型

本節主要介紹大型語言模型，在子節 2.2.1 中會介紹 LLM 的概論；子節 2.1.2 介紹大型語言模型的基礎架構。

2.2.1 大型語言模型概論

LLM 是生成式人工智慧模型的子項之一，是一種採用深度學習技術，用於自然語言理解與自然語言生成的新形態模型。LLM 主要是利用多種的大數據資料進行模型訓練，這些資料集包含了書籍、文章、網路資訊等各式樣的文字資訊。透過豐富的資料能夠讓 LLM 學習到自然語言的複雜性以及多樣性，進而更好的理解人類語言所代表的含意。

LLM 的結構是基於變換器模型 (Transformer) 的架構，這樣的架構讓 LLM 可以處理大量的文本資料，以及捕捉字彙間的關係。LLM 會透過預訓練 (Pre-training) 的技術對大量的文本進行訓練，預訓練可以讓模型學習到詞彙間的關係以及句子間的上下關係。在預訓練完成後 LLM 已經取得大部分現有的詞彙，但對於較專業之領域只透過預訓練並不能取得很好的效果。為了提高語言模型在特定任務上的處理能力，可以通過微調 (Fine-tuning) 來實現。微調是指利用特定領域的數據對 LLM 中的權重進行輕

微調整的過程。相對於重新進行預訓練所需的大量數據，微調只需要少量數據，尤其對於難以獲取數據的領域尤為重要。這種方法常應用於各種不同的領域，包括聊天機器人、語言翻譯和創意開發等。

近年來，許多 LLM 相繼被開發出來。例如，於 2018 年由 Google 推出的 BERT (Bidirectional Encoder Representations from Transformers) [5]，這是一種基於變換器 (transformer) 的雙向編碼器技術。BERT 模型的一大特點是能夠同時考慮到詞彙前後的上下文信息，從而生成更適切的詞彙表示。此外，BERT 採用了遮罩語言模型 (Masked Language Model: MLM)。在 MLM 設定中，系統隨機遮蓋句子中的某些詞彙，而 BERT 則需要預測這些被遮蓋的詞彙，這種方法有助於模型更好地學習詞彙間的相互關係，從而提升對語言的理解能力。

於 2019 年提出的 xLNet[6] 以 BERT 作為基礎進行改良，因在過往的處理自然語言時出現了模型無法處理長文本問題與忽視字詞順序性的問題。為了要處理長文本帶來的問題，在 BERT 中使用了絕對位置編碼 (Absolute Position Embedding: APE) 此編碼方式會導致模型無法考慮到詞彙間的關係，所以 xLNet 使用了相對位置編碼 (Relative Position Embedding: RPE) 可以捕捉到每個詞彙間存在的關係。另外 xLNet 使用了與 MLM 不同的技術置換語言模型 (Permutation Language Model: PLM) 作為訓練主體，透過重新編排打亂句子中詞彙的順序，讓模型學習更全面的前後文訊息。

於 2019 年提出的文本到文本轉換模型 (Text-to-Text Transfer Transformer: T5)[7] 是一種可以接受任一長度文本輸入，並生成相應文本的通用模型。因為 T5 將自然語言處理 (Nature language processing: NLP) 任務是為文本到文本的轉換任務。文本到文本的轉換模型通常使用單一的轉換器架構，其中包含了多層的注意力機制，而注意力機制可以使得模型關注不同位置的詞彙，所以此模型可以處理不同長度的文本輸入。

至今，廣為人知由 OpenAI 所開發的 Chat-GPT 基於 GPT 系列 [8] 與由 Facebook 所開發的 Llama 系列 [9]，此二模型是最為代表的 LLM。

2.2.2 語言模型基礎架構

語言模型 (Language Model, LM) 的架構示意圖如圖2.2所示，LM 的架構主要分為四個關鍵的部分：斷詞系統 (Tokenizer)、詞嵌入 (Word Embedding)、位置嵌入 (Position Embedding)、變壓器 (Transformer)。

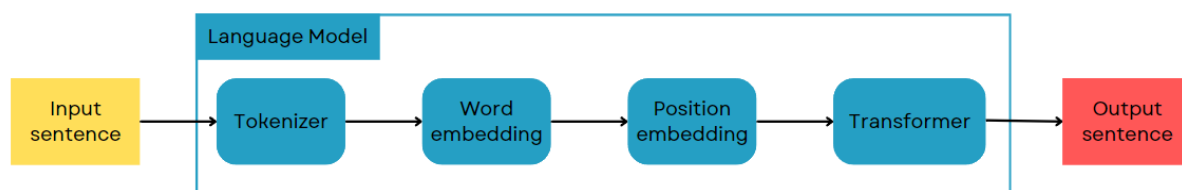


圖 2.2: 大型語言模型基礎架構圖

在 LM 中，主要的原理是參考輸入以及已經輸出的內容，並預測當下最適合的單詞。在 LM 的運作流程中，當使用者輸入一個句子後，LM 會首先利用斷詞系統將輸入的句子進行切割成單詞，如圖2.3所示。這些單詞被稱為令牌 (Token)，是 NLP 任務中處理的最小單元。接下來，這些令牌會通過一個嵌入層，被轉換成固定維度的向量形式。這一過程稱為詞嵌入 (word embedding)，它允許模型將離散的文本數據轉換為連續的向量空間中的點，從而便於進行各種數學操作。

舉例來說，假設一個嵌入層的輸出維度為 10 維，並且斷詞過後的句子長度為 9，則其輸入及輸出的關係如圖2.4所示。在這個例子中，每個令牌被轉換成一個長度為 10 的向量，而整個句子則被表示為一個 9×10 的二維矩陣。

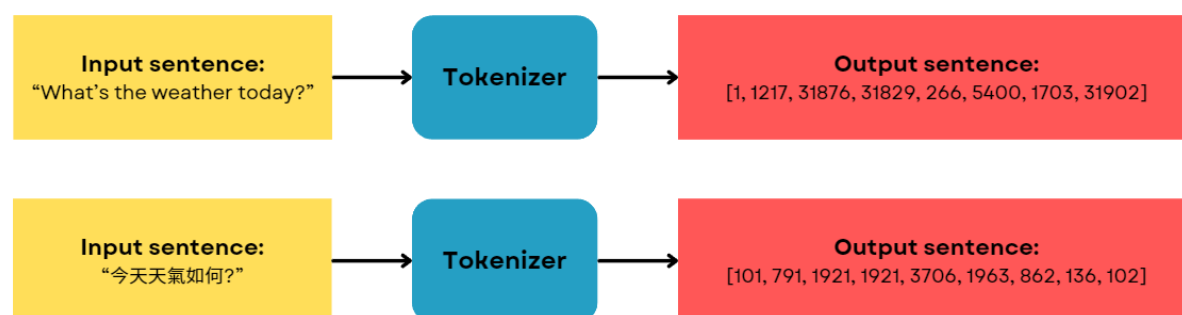


圖 2.3: 範例斷詞後的結果

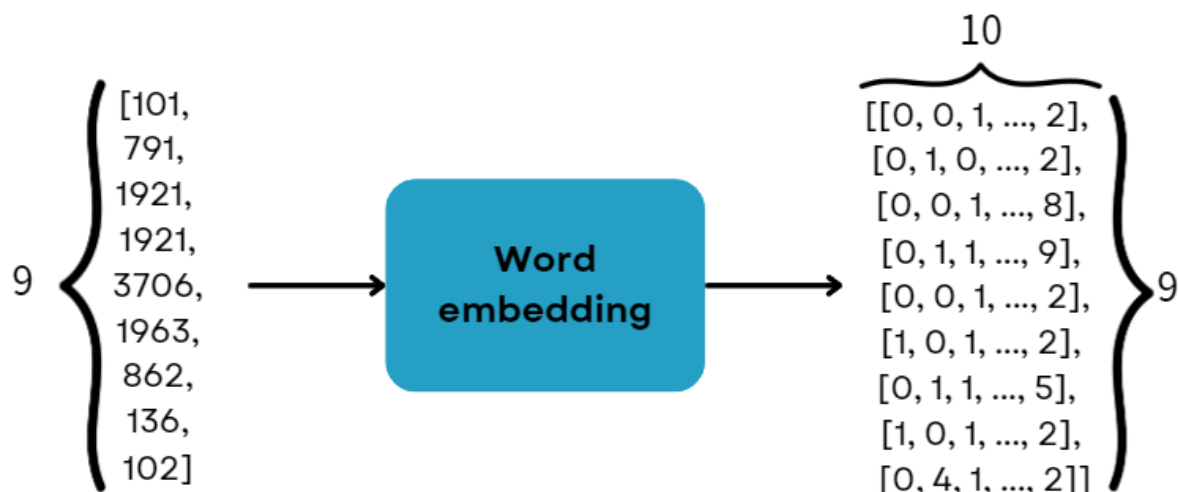


圖 2.4: Word embedding 範例

除了詞嵌入以外，位置嵌入 (Position Embedding, PE) 在語言模型中扮演著至關重要的角色，它讓模型能夠理解詞語在句子中的順序和位置關係。傳統的語言模型，如 RNN 或 LSTM，通過序列化的處理來捕捉前後文關係，但序列化的方法在長序列處理時會遇到效率和記憶體限制問題。引入 PE 後，模型如 Transformer 能夠並行處理所有詞語，PE 通過為每個 Token 加入一個獨特的位置向量，使得模型即使在缺乏明顯序列的情況下，也能識別出詞語的先後關係，從而保留語言的時序特性。

圖2.5呈現了 Transformer 的架構。編碼器 (Encoder) 以及解碼器 (Decoder) 是 Transformer 的主要核心架構，透過對資料的編碼以及解碼達到對資料的轉換。編碼器主要的功能是接收並且分析輸入的資料，然後將這些資料轉換成特徵向量，特徵向量可以捕捉到輸入資料的重要信息以及語境關係。而解碼器主要功能是基於編碼器所提供的特徵向量，逐步的生成輸出數據。

在編碼器以及解碼器中階包含了注意力機制 (Attention)[10]，此機制是 Transformer 的主要核心，而注意力機制主要核心在於對所有的內容同時進行平均加權運算，使得編碼器以及解碼器可以同時關注不同區塊的資料，其函式如下：

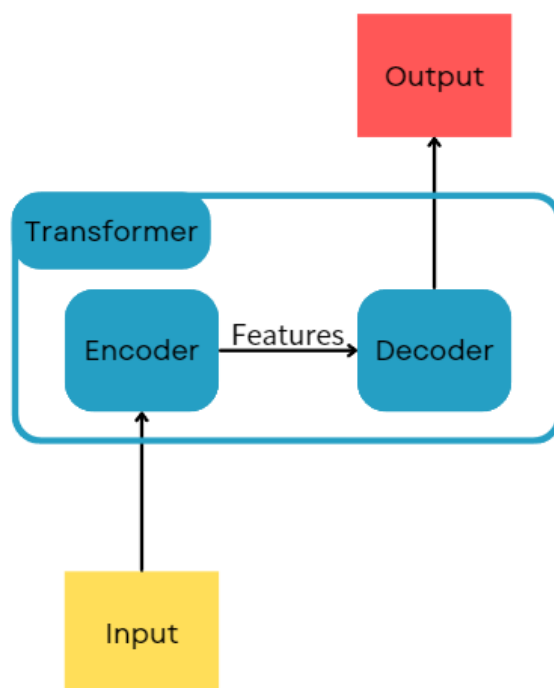


圖 2.5: Transformer 架構圖

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.1)$$

在注意力機制中， $Q \cdot K^T$ 的計算是至關重要的一步，它描述了當前被查詢元素（由查詢向量 Q 表示）與序列中其他元素（由鍵向量 K 表示）之間的關聯性。當這兩者的內積值較大時，表示兩者具有較高的相似度或關聯性，這通常意味著在給定語境下，這些元素在信息交流中應該被賦予較高的關注。

這些計算出的分數隨後會通過 Softmax 函數轉換成一個機率分佈，這個機率分佈代表了每個元素對最終輸出的貢獻程度。具體來說，Softmax 函數將分數正規化，使所有元素的貢獻度加起來等於 1，且較高分數的元素會獲得較大的權重，從而在生成時會更加被重視，而在 NLP 的領域中上述的「元素」指的就是「令牌」。

然而 Transformer 的結構中一定會包含編碼器以及解碼器，但在 LM 的架構中卻沒有規定需要編碼器與解碼器成對出現，正因為如此 LM 會衍生出多種的結構，例如由

OpenAI 所開發的 GPT 系列是只有使用解碼器作為架構的語言模型、由 Facebook 所開發的 Llama 是只有使用編碼器作為架構的語言模型。在不同的任務中會使用不同結構的語言模型，下列是各種語言模型適合的任務類型：

1. 只使用解碼器的模型：這類模型特別擅長於文本生成，如撰寫文章、創作故事和詩歌等。它們也常用於構建對話系統，提供自然流暢的對話回應。此外，這些模型在一定條件下也可用於語言翻譯和提供編寫程式輔助，如程式碼生成和解釋。
2. 只使用編碼器的模型：它們優秀於理解和分類文本。這包括進行情感分析、主題分類、命名實體識別，以及在問答系統中精確地從文本中提取答案。這些模型同時擅長評估文本片段間的語義相似性，有助於信息檢索和文本匹配。
3. 結合編碼器與解碼器的模型：主要用於機器翻譯，能在不同語言間進行高質量的翻譯。此外，這些模型也適用於文本摘要的生成，包括提取型和生成型摘要，並能有效處理圖像描述生成和視覺問答等多模態任務。

此外，NLP 領域中，位置嵌入主要分為絕對位置編碼 (Absolute position embedding, APE) 和相對位置編碼 (Relative position embedding, RPE) 兩種形式。絕對位置編碼的計算通常採用正弦和餘弦函數來對每個位置的每個維度進行編碼，這種方式可以保證即使在長序列中，位置信息也能被有效地捕捉和利用。具體的計算過程如下：

- 對於位置 pos 和維度 i ，位置編碼 $PE(pos, 2i)$ 和 $PE(pos, 2i + 1)$ 被定義為：

$$PE(pos, 2i) = \sin\left(pos/10000^{2i/d_{\text{model}}}\right) \quad (2.2)$$

$$PE(pos, 2i + 1) = \cos\left(pos/10000^{2i/d_{\text{model}}}\right) \quad (2.3)$$

其中 pos 是詞語在序列中的位置， i 是維度的索引，而 d_{model} 是模型的維度。

- 經過 word embedding 的向量會加上 position embedding，如同下表所呈現，使得

每一個維度中皆具有 position embedding。

pos	Updated Embedding
1	$[wordembed_{1,0} + PE(1, 0), wordembed_{1,1} + PE(1, 1)]$
2	$[wordembed_{2,0} + PE(2, 0), wordembed_{2,1} + PE(2, 1)]$

通過上述的位置編碼方式，Transformer 模型能夠在沒有任何循環層或卷積層的情況下，處理序列數據中的時間依賴性。使用這種方法，模型能夠識別位置的絕對感知，並且由於正弦和餘弦函數的周期性質，模型也能夠對很長的序列進行有效的學習。

相對位置編碼則依靠注意力機制來實現。相對位置編碼的引入是為了克服傳統注意力機制主要關注元素間內容相似性而忽視位置關係的缺陷。透過相對位置編碼，每個元素在計算注意力權重時不僅考慮內容相似性，還會根據它們之間的位置距離進行調整，這一策略顯著提升了模型對詞序和語法結構的理解能力。

實現相對位置編碼的具體步驟如下：

1. 定義相對位置關係：對於序列中的任意兩個位置 i 和 j ，定義它們之間的相對位置 $r = j - i$ 。
2. 生成相對位置編碼：為每個可能的相對位置 r 生成一個編碼向量 p_r ，這些向量可以是預先設計的或是在模型訓練過程中學習得到的。
3. 修改注意力分數計算：在計算 $Q \cdot K^T$ 的基礎上，將相對位置編碼得到的額外分數納入考慮。

$$\text{score}(i, j) = Q_i \cdot K_j^T + Q_i \cdot p_{j-i} \quad (2.4)$$

4. 應用 softmax 並計算最終輸出：將得到的分數使用 softmax 函數進行正規化，並透過這些權重對值 V 進行加權求和，以得到最終的輸出。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{\text{score}(i, j)}{\sqrt{d_k}}\right)V \quad (2.5)$$

這些方法的引入，使得模型在處理複雜的語言理解和機器翻譯任務時，能夠更準確地把握整體語境，提高了輸出的準確性和語境適應性。

圖2.6描繪了語言模型在生成句子時的推理流程。初始輸入句子經過斷詞、詞嵌入與位置嵌入處理後，生成三組向量：Q、K 與 V。這些向量被應用於當前最適合的令牌預測。這一預測過程涉及變壓器（Transformer），預測出的令牌將再次經過詞嵌入與位置嵌入，形成對下一個令牌的預測輸入。通過不斷重複此過程，語言模型能夠逐步構建出完整的句子。

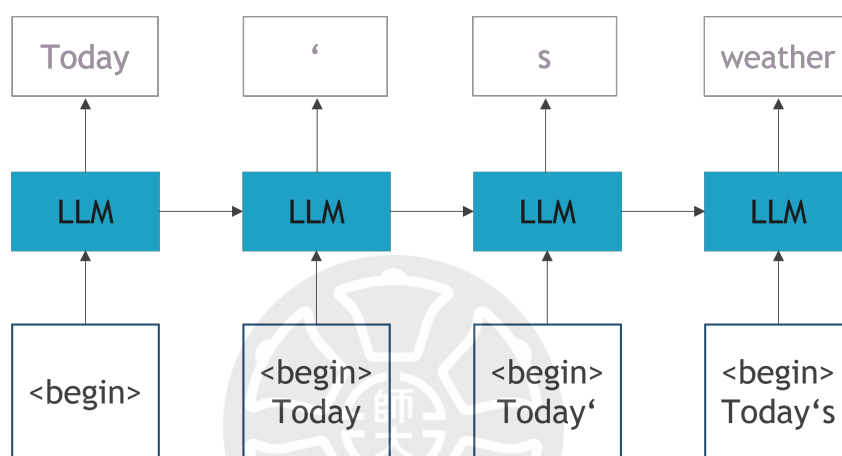


圖 2.6: 大型語言模型推理流程

2.2.3 微調概述

當前的學術研究中，許多研究團隊正致力於開發大型語言模型以提升其在特定領域的表現。其中，prompt engineering 是一種常見的方法，透過設定精確的用戶輸入指令，提高模型對問題的理解，從而獲得更佳的輸出效果。此方法不涉及對模型內部參數或結構的調整，而是透過優化輸入來達成目標。然而，此方法的有效性依賴於模型在預訓練階段已經接觸過相關的資料。對於模型未曾深入接觸的領域，僅憑 prompt engineering 往往難以達到預期的改進效果。

相較之下，微調（Fine-tuning）是另一種關鍵技術，它涉及對 LLM 內部參數的直接調整，以適應特定的應用需求。與 prompt engineering 相比，微調能夠對模型進行更

深層次的定制，但這也意味著更高的計算量和更大的硬體需求，這是微調實施中的主要挑戰之一。因此，微調通常由資源豐富的大型企業執行，他們能夠提供必要的計算資源和專業知識來進行大數據訓練和模型優化。

本研究專注於數學計算領域，數學計算領域在預訓練階段未能取得良好成效的情況下，僅透過 prompt engineering 也無法達成計算的目標。在本研究中，我們選擇了 LoRA (Low-Rank Adaptation) [11] 技術進行模型優化。這是一種先進以及有效的方法，通過對模型的關鍵權重進行局部訓練，用以精確控制模型行為，而無需進行大規模的權重訓練。LoRA 的應用不僅可以顯著降低對計算資源的需求，還能在不犧牲模型預訓練知識的前提下，有效適應特定的應用場景。這一技術特別適合於需要針對特定任務進行細節調整的情況，同時希望保留大部分預訓練時獲取的知識架構。其在自然語言處理領域的應用已經證明了其效果，對資源受限的研究環境尤為重要。

圖2.7呈現的是 LoRA 的技術概念圖，左邊藍色圖像為經過預訓練的權重，在未使用 LoRA 時會需要針對 W_0 中所有的權重進行調整。而使用 LoRA 時則不會對 W_0 進行調整，而是會額外加入 A 與 B 兩個神經網路，並且對 A 、 B 網路中的權重進行調整。其中 $r \ll d \wedge r \ll z$ ， b 為神經元中的偏置，若 W_0 、 A 與 B 皆為單層神經網路，並對 A 與 B 神經網路進行權重初始化如圖所示，則各別的權重數量如下：

$$\|W_0\| + \|b_{W_0}\| = d \times z + z \quad (2.6)$$

$$\|A\| + \|b_A\| = d \times r + r \quad (2.7)$$

$$\|B\| + \|b_B\| = r \times z + z \quad (2.8)$$

透過對 A 、 B 單層神經網路訓練，可以有效減少訓練時所需要調整之權重，以達到減少微調的計算量以及對於設備的限制要求。在本研究中，我們針對 Attention 中的 K 與 Q 投射中使用 LoRA 進行微調，使得有效並快速的學習元素間的關係。

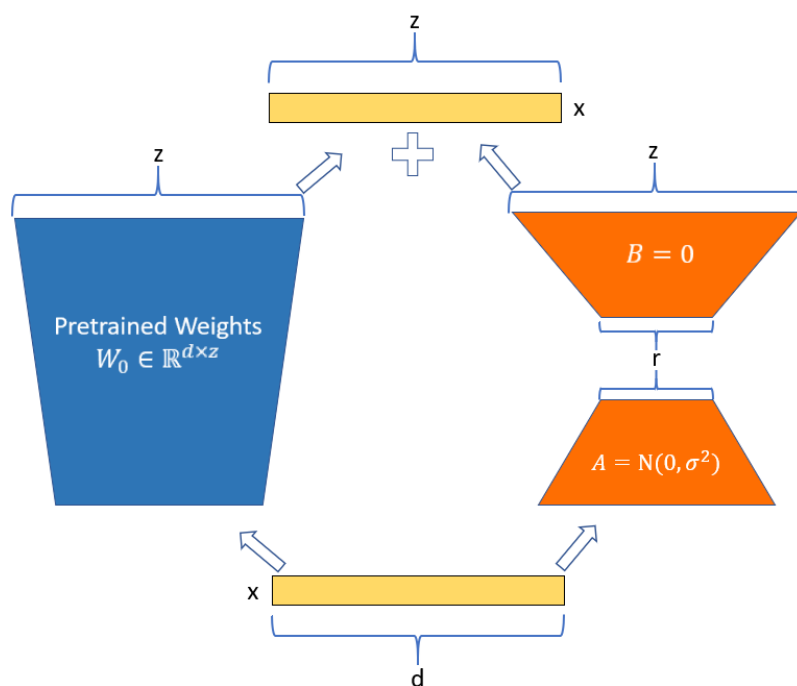


圖 2.7: LoRA 概念圖，其中輸入的維度為 d ，輸出維度 z ，以及 $r \ll d \wedge r \ll z$

2.2.4 在教育上的應用

LLMs 對於教育領域具有重要的影響，尤其顯著的是在提供個性化學習體驗和互動式教學上的應用。這些模型，如 GPT 系列，透過分析複雜的情境問題並提供適切的回饋，為學生提供即時的反應和互動，從而豐富學生學習上的體驗。這一功能對於各類學科，如語言學習、生物科學、物理、化學以及社會科學等，都顯示了巨大的潛力。

特別是在學術寫作和作業研究中，ChatGPT 等 LLM 已成為學生廣泛使用的工具。這些模型不僅能即時回答從數學問題到歷史事實的查詢，還能對學生的論文草稿提供批判性的分析，驗證信息的正確性，甚至提供改進的建議。此外，LLMs 能夠對長篇文章進行摘要和重點提煉，幫助學生快速掌握大量資訊，這對於需要涉獵廣泛文獻的高等教育學生尤為有益。

從教學實踐的角度看，LLMs 的應用不僅提高了學生的學習效率，也使教師能更精確地識別學生在學習過程中遇到的困難。這些模型提供的數據分析可以為教師調整教學計劃提供依據，使教學更具針對性和效果。例如，透過分析學生與 LLM 的互動記

錄，教師可以了解學生對某一學科概念的掌握程度，並根據需要進行個別化的輔導。

此外，LLMs 在自動化評分和反饋系統中的應用，極大地減輕了教師的工作負擔。這些模型可以自動批改學生的作業和考試，提供即時的反饋。這不僅提高了評估的效率，也增強了評估的一致性和客觀性。更進一步地，這種技術的應用可以支持更多創新的考試形式，如開放式問答和案例分析，這些形式通常需要更多的手工評分時間。

在教育領域中，LLMs 的兩種主要應用方式——作為教學輔助工具以及獨立的知識來源——各有其獨特的優勢和應用場景。作為輔助工具時，LLMs 能與專業知識系統整合，將複雜的專業知識轉化為學生易於理解的形式，提高學習的可接受性和效率。例如，學生可以直接用自然語言提問，LLM 則負責解釋並提供答案，這類交互對於提高學生的學習動機和參與度非常有效。

如圖2.8所示，Wolfram alpha 中允許使用者用自然語言作為輸入，而 LLM 在此地的功能為將自然語言轉換成 Wolfram Language，Wolfram Language 是 Wolfram alpha 資料格式化的方式，透過資料格式化使得內部的函數以及資料庫得以進行計算以及查找資料。這樣的方式不僅可以提高正確性透過查找資料庫或者是使用特定函數計算出答案，更可以做到控制生成數學圖形的精準度，這是單純使用 LLM 難以做到的部分。

作為獨立的知識來源時，如 Santa 這樣的系統被專門設計來輔助特定科目或技能的學習，如語言學習或專業考試的準備。這些模型能夠在教學的多個環節提供支持，包括教材的呈現、互動評估，以及模擬考試的設計。然而，雖然 LLMs 在處理廣泛的主題和基礎問題上表現出色，它們在某些專業或高度技術性的學科中可能無法完全替代專業教師的指導。

LLMs 在處理深度知識和專業技能方面存在限制主要是因為它們主要依賴於訓練數據中的模式識別。雖然這些模型能夠生成符合語境的回答，但它們的回答有時缺乏專業人士的深刻見解或基於最新研究的知識。此外，LLMs 在理解高度專業化的術語或解決複雜的問題時可能表現不佳，這可能導致它們在某些學術或專業領域的教學應用受

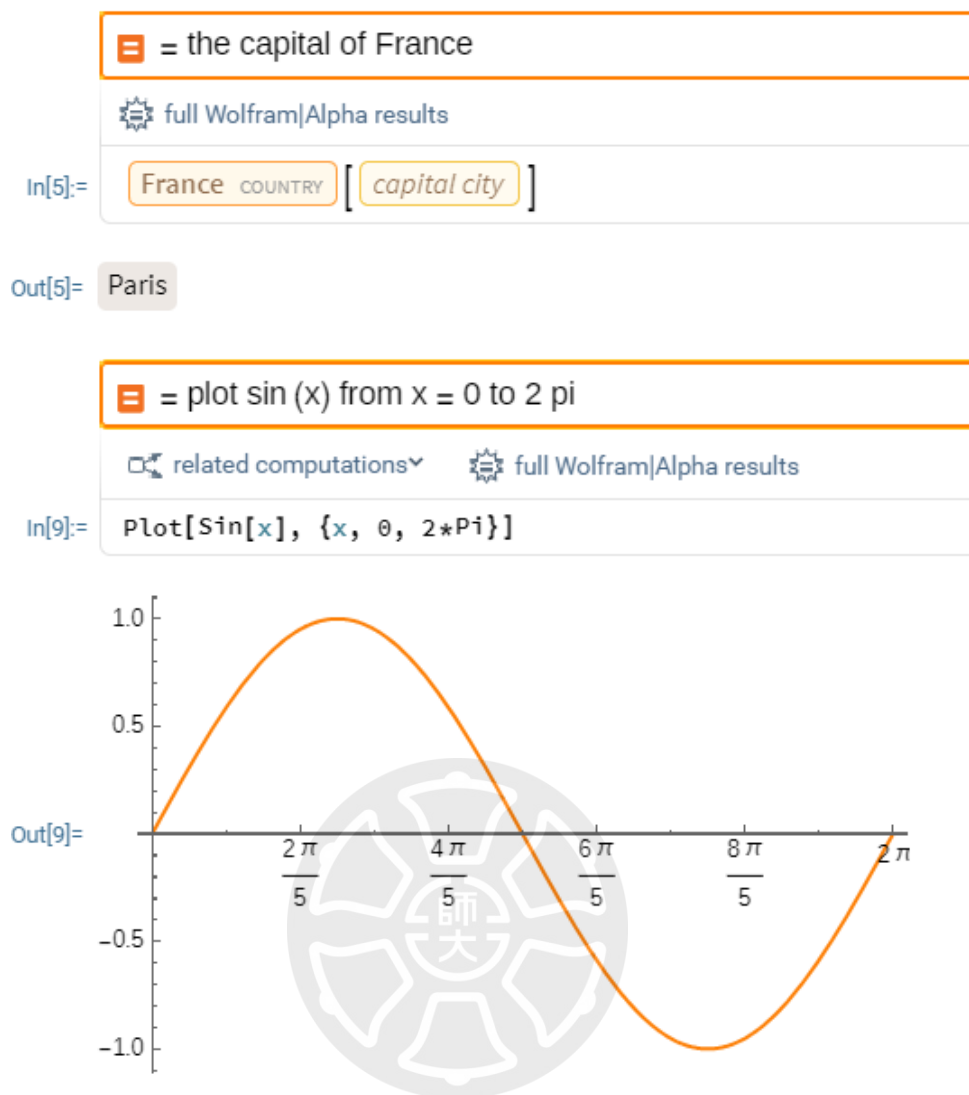


圖 2.8: Wolfram alpha 範例

限。

儘管如此，LLMs 的使用仍然具有顯著的優勢，特別是在激發學生的學習興趣和提供基礎知識方面。為了克服 LLMs 在教學中的局限性，可以將這些系統與專業教師的指導相結合，利用 LLMs 進行初始的知識傳遞和基礎問題解答，而更複雜的問題則交由專業教師處理。這種混合教學模式不僅可以提高教學效率，還可以確保學生能夠在需要時獲得專家級的指導和反饋。

因此，雖然 LLMs 目前在某些方面可能無法完全達到專業教育者的水平，它們作為教育工具的價值和潛力仍然巨大。未來的研究應著重於探索如何最有效地將這些先

進的技術工具與傳統的教學方法相結合，以達到最佳的教育成效。

2.2.5 在算術任務上的困難

研究表明，語言模型（LM）解決問題的能力隨著模型參數量的增加而提升。尤其是在大型語言模型（LLM）中，其擁有的龐大參數量使其解決問題的能力格外顯著。因此，許多教育平台和軟體紛紛將 LLM 作為主要的知識來源，向用戶提供廣泛的知識。然而，目前使用 LLM 的平台主要集中在教授語言和程式設計等學科，而將 LLM 用於數學教育的例子則相對稀少。

造成這樣的原因是 LLM 並不能很好的處理數學計算上的問題，儘管目前有眾多學者試著將 LLM 運用到數學類科的教學當中，其中有不少的研究是專門研究數學應用問題的解決，例如由 Microsoft 所提出的 Orac-Math[12]，此模型最主要是在學習如何解決國小數學的應用問題，以及學習邏輯上的推演。

然而造成 LLM 在數字計算領域能力低下的原因有兩個：

1. 數字令牌化: 對於語言模型來說令牌是計算的主要元素，對於語言領域來說儘管目前世界上存在著許多種語言，例如：中文、英文、西班牙文等等，而每一種語言中又會存在著許多不同的文字與詞彙，以現代漢語大詞典為例大約收錄了 30 萬的中文詞彙，儘管要定義一種語言中有多少的詞彙是困難的，但仍可想像其範圍。然而在數學計算中可能牽扯上千萬甚至無限大的數值計算，這對於模型要如何將數字令牌化是一個艱難的任務。如果將每一個數字視為一個令牌進行計算會造成輸入的維度無法計算；如果將每一個數字去作拆解成數個令牌進行計算，要如何定義拆解的方式又是另外一個艱難的問題。
2. 位置嵌入: 在 2.2.2 小節中有說明關於位置嵌入的部分，位置嵌入對於有順序性的資料內容來說具有相當大的幫助。通常在自然語言中越靠近的令牌對當下的生成會有比較大的影響，但在數學領域中卻不是如此，例如： $12+35=47$ ，我們可以發

現數字 7 與數字 4 在實際上是沒有任何關聯性的，數字 7 反而與數字 2 和 5 有強烈的關聯性，所以對於數學計算任務中位置嵌入的形式可能會造成計算低下的問題。

每一個模型在斷詞系統或者是位置嵌入的方式都不盡相同，儘管使用相同的數據集進行訓練，結構上的不同會造成訓練結果也會存在著差異。

2.3 針對算術任務微調的大型語言模型

儘管 LLM 在算術任務上存在著巨大的困難，仍有不少的學者致力於提升 LLM 在數學算術任務上的能力。GOAT(GOod at Arithmetic Tasks)[3] 是一個透過監督式微調的大型語言模型，目的為提高 LLM 在算術任務解答的正確性，因為 LLM 對於算術任務中的加法與減法任務具有學習性，然而對於乘法以及除法任務不具有學習性。

GOAT 為了要縮減此任務的複雜程度，選擇使用思維鍊 (Chain of Thought, CoT)[13] 的方式給予模型固定化的計算流程，例如：當 LLM 在遇到乘法任務時，會試著將數值轉換成容易計算的型式，如圖2.9所示 GOAT 在計算乘法時會將數值進行拆解，GOAT 在進行乘法運算時會選擇將較小的數值進行處理，將較小的數字分割成個位、十位、百位等，這樣的方式可以確保在進行乘法計算時乘數都只有單一位元需要被計算，而不需要進行兩位數或者是兩位數以上所需要的加法計算。

圖2.10中顯示此 GOAT 在面對除法時使用的計算方法，可以看出是透過利用了與人類計算方法相似的直式除法概念將問題逐步進行計算，並且在計算的過程中導入了乘法的計算，最終在將計算過程中得出的答案進行整合得出商數以及餘數，這種方式確實學習了人類計算數學除法時的處理模式。儘管 GOAT 只有處理兩個數值間的運算，但這種透過 CoT 方法提高計算正確率的技術也影響了後續的研究。

[14] 提出了透過相對位置嵌入讓 LLM 可以處理長度泛化問題，使得 LM 能夠處理

乘法以及除法的問題，不需要透過 CoT 的形式。[4] 中將 GOAT 與 [14] 所提到的方法進行整合，與 GOAT 不同的地方在於針對乘法以及除法的任務不需要透過 CoT 的形式，並且可以處理多數之間的運算。[4] 中使用了 CoT 的技術以處理多數字的運算，在他們的實驗中發現在一個計算中同時計算多個數字時容易造成問題，所以為了要減少錯誤率的發生他們在每次的運算中只會考慮兩個數字間的計算。圖2.1中呈現此模型在運算多數值的過程，首先若問題中的操作子是同一運算次序時，則會從左邊開始進行計算; 若問題中的操作子具有不同運算次序時，則會優先處理優先次序之運算。雖然此研究中並未考慮數值間的關係，而是單純的以排列之順序進行計算，但也展現了 LM 對多數問題的解題能力。

Instruction	Output
$1+8/1*10+2$	$1+8/1*10+2 = 1+8*10+2 = 1+80+2 = 81+2 = 83$
$5+5+8/1*2$	$5+5+8/1*2 = 5+5+8*2 = 5+5+16 = 10+16=26$
$3/2/7$	$3/2/7 = 1.5/7 = 0.21428571428571427$

表 2.1: 在 MathGLM 中用於訓練之範例

Instruction	Output
276 * 918	$276 * 918$ $= 918 * (200 + 70 + 6)$ $= 918 * 200 + 918 * 70 + 918 * 6$ $= 183600 + 64260 + 5508$ $= 247860 + 5508$ $= 253368$
58965017 x 3028	$2292 * 48815241$ $= 48815241 * (2000 + 200 + 90 + 2)$ $= 48815241 * 2000 + 48815241 * 200 + 48815241 * 90 + 48815241 * 2$ $= 97630482000 + 9763048200 + 4393371690 + 97630482$ $= 107393530200 + 4393371690 + 97630482$ $= 111786901890 + 97630482$ $= 111884532372276 * 918$ $= 918 * (200 + 70 + 6)$ $= 918 * 200 + 918 * 70 + 918 * 6 = 183600 + 64260 + 5508 = 247860 + 5508$ $= 253368$
58965017 x 3028	$58965017 * 3028$ $= 58965017 * (3000 + 20 + 8)$ $= 58965017 * 3000 + 58965017 * 20 + 58965017 * 8$ $= 176895051000 + 1179300340 + 471720136$ $= 178074351340 + 471720136$ $= 178546071476$

圖 2.9: GOAT 在乘法問題的處理方式
 'Instruction' 為模型之輸入，'Output' 為模型之計算之過程。

Instruction	Output
Solve $323894723/7$ and give the calculated value.	$323894723 / 7 = 46270674 \text{ R } 5$
$542764255 / 84635$	$542764255 - 84635 * 6000 = 542764255 - 507810000 = 34954255$ $34954255 - 84635 * 400 = 34954255 - 33854000 = 1100255$ $1100255 - 84635 * 10 = 1100255 - 846350 = 253905$ $253905 - 84635 * 3 = 253905 - 253905 = 0$ Therefore, $542764255 / 84635 = 6413$
What is the numerical outcome of $9293195368/91639$?	$9293195368 - 91639 * 100000 = 9293195368 - 9163900000 = 129295368$ $129295368 - 91639 * 1000 = 129295368 - 91639000 = 37656368$ $37656368 - 91639 * 400 = 37656368 - 36655600 = 1000768$ $1000768 - 91639 * 10 = 1000768 - 916390 = 84378$ Therefore, $9293195368 / 91639 = 101410 \text{ R } 84378$

圖 2.10: GOAT 在除法問題的處理方式
'Instruction' 為模型之輸入，'Output' 為模型之計算之過程，其中 R 代表計算的餘數。

第三章 微調設計

本章節中，本論文將介紹本研究所提出的微調設計，其中包括大型語言模型的斷詞系統、針對於此任務之數據集設計。在本章之探討方向皆會影響 LLMs 在處理數學算術任務上的準確度。在生成多種解答的大型語言模型中，我們所提出的方法可以讓模型在數學領域中維持 LLM 的多樣性的特點，應用設計之方式可以提升大約 10% 的平均正確率。

圖3.1是本研究之系統架構。首先使用者可以輸入數學算術問題，接著 LM 會對該問題進行一次的推理，而在一次推理中會生成多種結果，每種結果是對問題進行下列的其中一種操作：

1. 雙數值之加法與減法：在先前的研究中發現 LM 在雙數字之運算具有高度的學習性，但若提升同時運算之個數時則會造成正確率下降，所以本研究在處理加減時則採用雙數值運算原則。
2. 函數之運用：儘管先前的研究中展現了 LM 在乘法以及除法的計算能力，但本研究並非旨在提升模型在算術方面的能力以及單純的得出答案，而是要提供多種解法，所以為了要避免 LM 在計算上的錯誤，我們設計了多種函數供 LM 使用，使得模型可以例外用部的系統進行計算。
3. 針對數字拆解：對於人類而言在計算數學問題時並非一昧地將所有的數值做計算，而是會透過各種計算的技巧簡化計算之複雜度，其中數字的拆解是最為常見的操作，所以 LM 會學習如何利用數字拆分的技巧讓計算的過程變得簡單。



圖 3.1: 本研究之系統架構

3.1 斷詞方式探討

在大型語言模型中，斷詞系統扮演著相當重要的部分，可以說斷詞的優劣會影響到 LM 對於任務的回答的能力。表3.1呈現了 GPT 與 Llama 在英文斷詞上的表現，以及在中文斷詞中使用了 BERT 與 Xlnet 作為範例，我們可以發現在英文斷詞中 GPT 與 Llama 兩者的斷詞結果的差距並不大，反觀在中文斷詞中 BERT 與 XLnet 兩者的斷詞結果有巨大的差距。

表 3.1: 在不同斷詞系統下的斷詞結果

	Input	GPT	Llama
英文	How's the weather today?	[How, 's, the, weather, today, ?]	[How, ', s, the, weather, today, ?]
	Input	BERT	XLnet
中文	今天天氣如何?	[今, 天, 天, 氣, 如, 何, ?]	[今天, 天氣, 如何, ?]

除了中文的斷詞有如此的差異之外，對於數字上的斷詞也是各有不同。如同前面所提，數學計算在深度學習領域所遇到的問題是模型的參數量是固定的，而數字有著無限多種可能性。所以要如何利用有限的資源去表示無限多的數字成為了一大難題。

而在本次的研究中我們專注於比較兩種不同的斷詞方式對於算術任務上的影響。

Input	12345		123456	
Model	GPT	Llama	GPT	Llama
Tokenization	[123, 45]	[1, 2, 3, 4, 5]	[123, 456]	[1, 2, 3, 4, 5, 6]
Input	321000		320000	
Model	GPT	Llama	GPT	Llama
Tokenization	[321, 000]	[3, 2, 1, 0, 0, 0]	[3, 200, 00]	[3, 2, 0, 0, 0, 0]

表 3.2: 此表呈現在 GPT 以及 Llama 兩種斷詞系統下的數字的斷詞結果

如表3.2所示，在 Llama 的斷詞系統中會將所有的數字中的每一個元素做切斷；在 GPT 的斷詞系統中會出現多種情況，可以發現被切割的數值長度可能是 1 到 3 元素，儘管 321000 與 320000 具有相同的長度，但兩者的分割方式卻有著不同之處。

在相同數字長度的情況下，Llama 的斷詞結果具有一致性，每一個 token 皆為一個數字或者符號；GPT 的斷詞結果比較難以預測，具有相同長度的數值也有可能被切割成不同長度的 tokens，儘管 GPT 的結果可以利用較少的 token 數量表達相同的數值，但我們認為不同的斷詞方式可能會造成計算上的困難。在第四章會對兩種斷詞系統進行實驗以及比較。

3.2 數據集設計

在本研究中數據集皆利用 Python code 進行生成。在本研究中我們專注於探討 LLMs 對於數學算術任務是否有能力生成多種解答，因為在算術任務中一個問題會存在多種可以解答的方式，而我們想要的是盡可能推演出不同的解答，所以我們這邊並非使用單一回應生成，而是多解答生成。本節將會介紹我們所提出的 3 種設計方式，這些設計方式在提升此研究之效能上有顯著的幫助。

3.2.1 數學運算代理

從 2022 年起，GPT 模型不僅被廣泛用作數據查詢的搜索器和編程助手，也開始輔助處理數學計算任務。通過編程直接調用外部系統執行計算，LLMs 在處理數學問題上的正確率得到了顯著提升。本文借鑒相同的方式，允許模型在執行基礎運算時調用外部計算系統 (Computer Algebra System, CAS)。如圖 3.1 所示，模型判斷需要使用 CAS 時，可以通過特定的函數調用進行運算。

與過往的研究的不同之處在於本研究並非全權使用 CAS 系統進行計算，而是只在 LM 難以解決的部分做使用。為應對大數字運算時迭代次數過多的問題，我們設計了運算符供 LLM 使用。例如，表達式 12×45 會被轉化為 `mul(12,45)`，由模型自動判斷是否替換運算符以及運算的參數，從而簡化計算過程並減少錯誤。為解決乘法與除法的計算準確率問題，我們設計了兩個專門的函數：乘法 (`mul`) 和除法 (`div`)。這些函數允許模型目前設定接受兩個參數的計算，簡化了與 CAS 系統的互動。如表 3.3 所示。

在表 3.3 的第一個範例中，可以發現 LLM 在計算乘法問題時，會分為兩種不同的方式，第一種是對數值進行拆解讓數值變得容易運算，第二種方式則是透過使用我們設計的函數進行運算，並且自行去判斷是哪些數值需要被外部系統進行計算然後回傳已經被替換完成的解答。在除法方面則比較直觀，模型仍會對數值進行操作讓問題變得比較簡單，例如第二個範例中進行結合律。但若是需要對除法進行計算時則會透過外部系統進行計算。

LM 可以在解題的過程中使用這些設計好的函數，但並非在所有遇到乘法以及除法時都必須要使用這兩種函數才能進行解題，若 LM 在判斷運算內容較為容易時可能也會自行透過推理的方式計算出問題的解答。例如： $20 \times 534 = 10680$ ，則模型可以透過內部推理的形式直接計算出答案，因為對於 LLM 而言這種較為簡單的運算是可以透過訓練的方式學習得到。在乘法運算中，外部系統會辨識 LM 之輸出並計算出被轉換之部分，輸出為兩個參數的乘積；在除法運算中，外部系統會評估除數與被除數之間的關

係，若能整除則返回整數值，否則返回最簡分數。如表3.4所示。

此外，這種模型的設計允許更靈活地處理各種數學問題，特別是那些涉及高度複雜運算的情況。通過結合 LLM 的強大推理能力和外部計算系統的精確運算能力，我們能夠提供一個更為全面且效率高的解決方案。

	Original	Function
Output	$377*499*527-827+213 =$ (A) $(400-23)*499*527-827+213$ (B) $377*499*527-614$ (C) $377*(500-1)*527-827+213$ (D) $377*499*(500+27)-827+213$ (E) $188123*527-827+213$	$377*499*527-827+213 =$ (A) $(400-23)*499*527-827+213$ (B) $377*499*527-614$ (C) $377*(500-1)*527-827+213$ (D) $377*499*(500+27)-827+213$ (E) $\text{mul}(377,499)*527-827+213$
Output	$508/240-623/240 =$ (A) $508/240-\frac{623}{240}$ (B) $(508-623)/240$ (C) $\frac{623}{240}-623/240$	$508/240-623/240 =$ (A) $508/240-\text{div}(623,240)$ (B) $(508-623)/240$ (C) $\text{div}(508,240)-623/240$

表 3.3: 此表呈現使用 Function 的資料集設計方式

Original 表示未對資料集作任何更動，Function 表示將乘法的部分替換成自訂義函數。此表中有兩個問題分別是乘法以及除法問題，Input 代表輸入的問題，Output 代表 LM 的輸出。

Input	Output
$\text{mul}(377,499)*527-827+213$	$188123*527-827+213$
$508/240-\text{div}(624,240)$	$508/240-\frac{13}{5}$
$508/240-\text{div}(624,312)$	$508/240-2$

表 3.4: 此表為 CAS 系統之輸入以及輸出

3.2.2 答案生成的問題關聯控制

在大型語言模型的架構中，包含了斷詞與模型內部推理，在上一個小節中我們探討了不同的斷詞系統對於次任務的影響，這一小節會著重在內部推理，而內部推理通常包含了 Encoder layers 或 Decoder layer，在這兩種 layer 中皆具有多頭注意力機制 (Multi-Head Attention) 是在 LLM 中不可或缺的部分，多頭注意力機制最主要的功能是可以讓模型同時去關注所有位置上的序列元素，這樣的機制可以幫助模型更好的去了解整個序列的內容。

語言模型在進行 token 迭代推理 (Inference) 的時候會考慮提示 (Prompt) 以及已經被生成的 tokens，這件事情讓模型有辦法去理解自然語言中文字的排列性質，以及輸出有邏輯的句子結構。儘管我們認為注意力機制可以避免大型語言模型在推理其他答案時，透過計算 Q、K、V 去判斷輸入之 tokens 對於答案的影響程度進而去生成最適合當下的 token，但經過實驗發現經過微調後的模型在此任務上的能力仍表現不佳，所以我們針對大型語言模型生成之答案進行分析，找尋可能造成正確率低下的主要原因。

在深入分析過程中，我們發現影響答案品質的一個顯著因素是答案生成的順序。具體來說，在大型語言模型生成答案的過程中，早期生成的答案通常具有較高的正確率，而隨著答案生成順序的推移，後續生成的答案錯誤率顯著增加。這一現象可能與模型在生成過程中過度依賴先前已生成的 tokens 有關，這些先前的 tokens 可能會對模型的當前判斷造成誤導。

為了排除複雜的模型設計導致錯誤增加的可能性，我們進行了一系列控制實驗。結果表明，即使在簡化了計算流程的情況下，後續生成答案的錯誤率仍然較高。基於這些發現，我們推測，模型在生成每一個新的 token 時，都會參考之前已生成的答案，這種累積參考的機制可能在不斷地放大初期的誤差。

為了解決這一問題，我們嘗試了一種新的輸出策略，即在生成答案時，每次都重新關聯到原始問題，以減少解答累積的影響。具體實施方式如下，我們修改了語言模

型的輸出格式，使得模型在生成每個答案時，都將問題作為參考點重新呈現。例如，在解決數學問題的場景中，如表3.5所示，每個答案的生成都會伴隨著原問題的提示，這有助於模型更加精確地定位信息並提高答案質量。通過這種方式，每個答案的生成都能保持對原始問題的高度關聯性，從而提高模型對問題的理解深度並減少生成錯誤答案的機會。

Input	377*499*527-827+213	
Output	Original	$377*499*527-827+213 =$ (A) $\text{mul}(377,499)*527-827+213$ (B) $377*499*527-614$ (C) $377*\text{mul}(499,527)-827+213$ (D) $\text{mul}(377,527)*499-827+213$
	Relevance Control	(A) $377*499*527-827+213 = \text{mul}(377,499)*527-827+213$ (B) $377*499*527-827+213 = 377*499*527-614$ (C) $377*499*527-827+213 = 377*\text{mul}(499,527)-827+213$ (D) $377*499*527-827+213 = \text{mul}(377,527)*499-827+213$
Input	508/240-623/240	
Output	Original	$508/240-623/240 =$ (A) $508/240-\text{div}(623,240)$ (B) $(508-623)/240$ (C) $\text{div}(508,240)-623/240$
	Relevance Control	(A) $508/240-623/240 = 508/240-\text{div}(623,240)$ (B) $508/240-623/240 = (508-623)/240$ (C) $508/240-623/240 = \text{div}(508,240)-623/240$

表 3.5: 此表呈現使用 Relevance-Control 的資料集設計方式

Output 表示是 LM 之的輸出。Relevance-Control 是重新對 Oringal 重新編排，將問題編排進每一個解答之中。

3.2.3 答案生成的序列控制

本節將介紹我們提出的一種新的資料集設計方法，旨在提高生成正確率。在前一節中，我們討論了大型語言模型（LLM）在執行特定數學任務時遭遇的兩大問題及其對應的解決方案。首先，針對大型語言模型在執行乘法與除法運算時呈現出的高錯誤率，我們提出一種策略，允許模型自主判斷何時引用外部系統進行計算，並指定相關參數。其次，由於大型語言模型的結構特性會考慮已生成的答案，這些答案往往不利於後續其他答案的生成。對此，我們通過重新設計輸出格式，使模型在生成每一個答案時都能重新考慮初始問題，從而優化答案的質量。

正如眾所周知，當使用大型語言模型解答問題時，若希望得到重點明確或更詳細的回答，可通過更精細的需求描述或採用列點方式來增強模型的處理能力。這是因為模型在獲得更多資料時，能夠從中提取更多特徵，以滿足人類的需求。語言領域的豐富性賦予了 LM 強大的特徵識別能力，使其能夠迅速通過 token 找到關鍵特徵並做出回應。這也解釋了為何我們通常只需提供一句話，LM 便能大致推測出問題的核心。

然而，在數學計算任務中，由於斷詞階段已經大幅減少 token 的種類，如數字 0 至 9 和其他運算符號，而且數字間的 token 基本上不具相關性，因此在這類任務中模型所擁有的特徵相對匱乏。鑑於本研究旨在生成多樣化的解題路徑，特徵的匱乏和答案的高度開放性往往導致誤答。為了解決這些問題，我們決定增加提供的特徵數量。考慮到實際應用場景，我們避免在輸入端額外增加信息以提升模型效能，最終選擇讓 LM 自行生成所需的特徵。

在 LLMs 中具有位置編碼（Position Embedding），這個技術讓模型能夠捕捉到具有順序性的語言資料中的特徵，以及輸出能夠讓人類理解之句子其中也包含了順序。我們認為模型在生成數學計算任務的解答時也需要有一定的順序性，例如使用 CoT 按照順序進行計算，所以我們對輸出的內容作了一定的排序如表3.6所示。並且認為大型語言模型生成解答時一律是從左到右依序進行生成，所以在我們設計的解答中每一個

問題所對應的解答們的順序會依照數值在整個問題中的位置進行排列。

原始資料集的輸出解答可能會隨機地從任何數值開始操作，而採用序列控制後的排序確保了一致的處理順序。這種方法可以使得模型在生成解答時可以自行判斷問題中應該要處理的部分，利用順序的概念使得 LLM 能夠生成隱性的特徵。當我們採用這種方法時，模型在生成解答時可以更好地理解並遵循問題中的數值順序，從而提高解答的準確性和可靠性。這不僅提升了模型對數學問題的處理能力，也讓生成的答案更加符合數學計算的規範。

表3.6，Original 部分表示的是資料集中的解答是隨機排序。Sequence Control 的部分表示資料集中的解答順序是有經過特定標準進行排序的，排序的標準為從表達式的左邊開始進行操作，逐步向右尋找可以操作之運算，以及函數使用的部分會作為最後



的輸出。

Input	$(118+185)/(894+868)-470$	
Output	Original	$(118+185)/(894+868)-470 =$ (A) $(118+185)/1762-470$ (B) $118/(894+868)+185/(894+868)-470$ (C) $303/(894+868)-470$
	Sequence Control	$(118+185)/(894+868)-470 =$ (A) $303/(894+868)-470$ (B) $(118+185)/1762-470$ (C) $118/(894+868)+185/(894+868)-470$
Input	$913-752*913$	
Output	Original	$913-752*913 =$ (A) $913-752*(900+10+3)$ (B) $913*(1-752)$ (C) $913-\text{mul}(752,913)$ (D) $913-(700+50+2)*913$
	Sequence Control	$913-752*913 =$ (A) $913-(700+50+2)*913$ (B) $913-752*(900+10+3)$ (C) $913*(1-752)$ (D) $913-\text{mul}(752,913)$

表 3.6: 此表呈現使用 Sequence control 的資料集設計方式

第四章 實驗結果與分析

本章節將介紹經過上述微調方法設計之實驗結果以及分析方法的成果，在本章的第 1 節中會介紹用於衡量本任務成功機率的方法，第 2 章節中會介紹如何衡量在經過微調後模型生成答案之多樣性，第 3 小節中則會進行消融實驗，以及第 4 小節中會著重分析本任務中存在的問題以及解決之方法。

本章節實驗中存在著多個用於微調的資料集，儘管資料集間略有不同，但不同之處只在於 Output 形式，在問題的種類以及解題的方式並沒有任何的不同，所有的資料集皆具有 51,000 筆輸入-輸出對。在本章節實驗中所使用之問題集共 600 題，是由加減乘除四種運算符號進行組合的數學算術問題，以及問題中數值的數量為 2-7 個數值不等。

4.1 正確率

為了全面衡量模型在多變情境下提供解決方案的正確性，本研究引入了平均正確率（Average Accuracy）這一綜合性指標。此指標不僅反映了模型在廣泛應用情況下的整體性能，而且提供了一個直觀的方法來評估模型對不同問題的一致性和可靠性。

平均正確率的主要目的是評估模型在提供多個可能解答時的表現，這對於確定模型能否穩定地重現正確答案至關重要。通過對各個測試案例進行正確性評估，可以有效識別出模型在哪些方面表現優異，以及哪些方面需要進一步調整或改進。這項研究中對於正確解答的定義如下：對原始問題進行雙數值之加法與減法、函數運用，以及

針對數字拆解的三種操作，若解答未能正確使用這些操作或重複使用它們，則判定為錯誤解答。

在實際應用中，針對每個測試案例的單個正確率 Accuracy_i 定義如下，其中 n_{correct} 代表模型在該實例中給出的正確答案數量，而 n_{total} 則是模型在該實例中給出的總答案數量：

$$\text{Accuracy}_i = \frac{n_{\text{correct}}}{n_{\text{total}}} \quad (4.1)$$

基於此，平均正確率 Average Accuracy 的計算方式如下，其中 N 表示測試中涉及的問題總數， Accuracy_i 則表示第 i 個問題的正確率：


$$\text{Average Accuracy} = \frac{1}{N} \sum_{i=1}^N \text{Accuracy}_i \quad (4.2)$$

此公式確保了在整體評估過程中，每個單獨問題的重要性被均等考慮，從而允許研究人員全面瞭解模型在不同類型問題上的綜合表現。

4.1.1 斷詞系統效能對比

在前一章節我們探討了段詞系統會對自然語言生成任務在成一定程度上的影響，所以我們在此小節中使用了兩種不同斷詞系統用於訓練模型。為了要更了解斷詞系統對於數學算術任務的影響，所以在微調的過程中，我們固定使用 Llama 結構作為模型基礎架構，以及兩者皆使用同一份訓練資料。在斷詞系統上分別使用了3.1節所提及的兩種斷詞系統，分別是 GPT 以及 Llama 的斷詞系統。

圖4.1呈現分別 2 種不同斷詞方式的進行訓練的結果，這兩種斷詞的方式分別為：

1. Method1: 對數值進行切割，但並未按照明顯的固定方式，如同 GPT 的斷詞方式。
2. Method2: 對數值進行平等的切割，如同 Llama 的斷詞方式。

縱軸表示了模型在處理測試問題資料集後所計算出的平均正確率，而橫軸則顯示了學習步驟 (Learning Steps)。通過這張圖表，我們可以直觀地觀察到模型在經歷不同學習階段後，對問題解答的平均正確率變化情況。在訓練初期階段的觀察中，我們發現兩個模型在平均正確率上已經表現出約 10% 的差異，這在本任務中代表著一個顯著的性能差距。

隨著學習的深入，即隨著更多學習步驟的完成，我們觀察到兩個模型在解決此類數學任務的能力均有所提升，這也間接證明了在 LM 訓練過程中，此類數學任務的解決策略是可以被有效學習的。具體來說，經過一系列的學習步驟後，採用 Method2 的模型在平均正確性上比採用 Method1 的模型表現更佳，這表明 Method2 在處理算術問題上可能具有更優的策略和方法。

值得注意的是，在整個訓練過程中，我們可以觀察到兩種模型之間的平均正確率始終保持一定的差距。無論在何種學習階段，使用 Method1 的模型在效能上均未能超越使用 Method2 的模型。這一現象表明，模型中的斷詞策略對算術任務的解決具有顯著影響。Method2 採用的對每個數字進行平均切割的方法，能夠更有效地提升語言模型在處理複雜數學計算任務時的性能。

綜上所述，我們的研究結果強調了在模型訓練過程中選擇合適的斷詞方法的重要性，尤其是在面對需要精確數學解答的 NLP 任務時。

4.1.2 資料集設計效能對比

在3.2節中，我們提出了 3 種對數據集修改的方式，我們認為這 3 種方式對於本研究的都具有一定的幫助，所以接下來會詳細探討這 3 種方式對於本任務的影

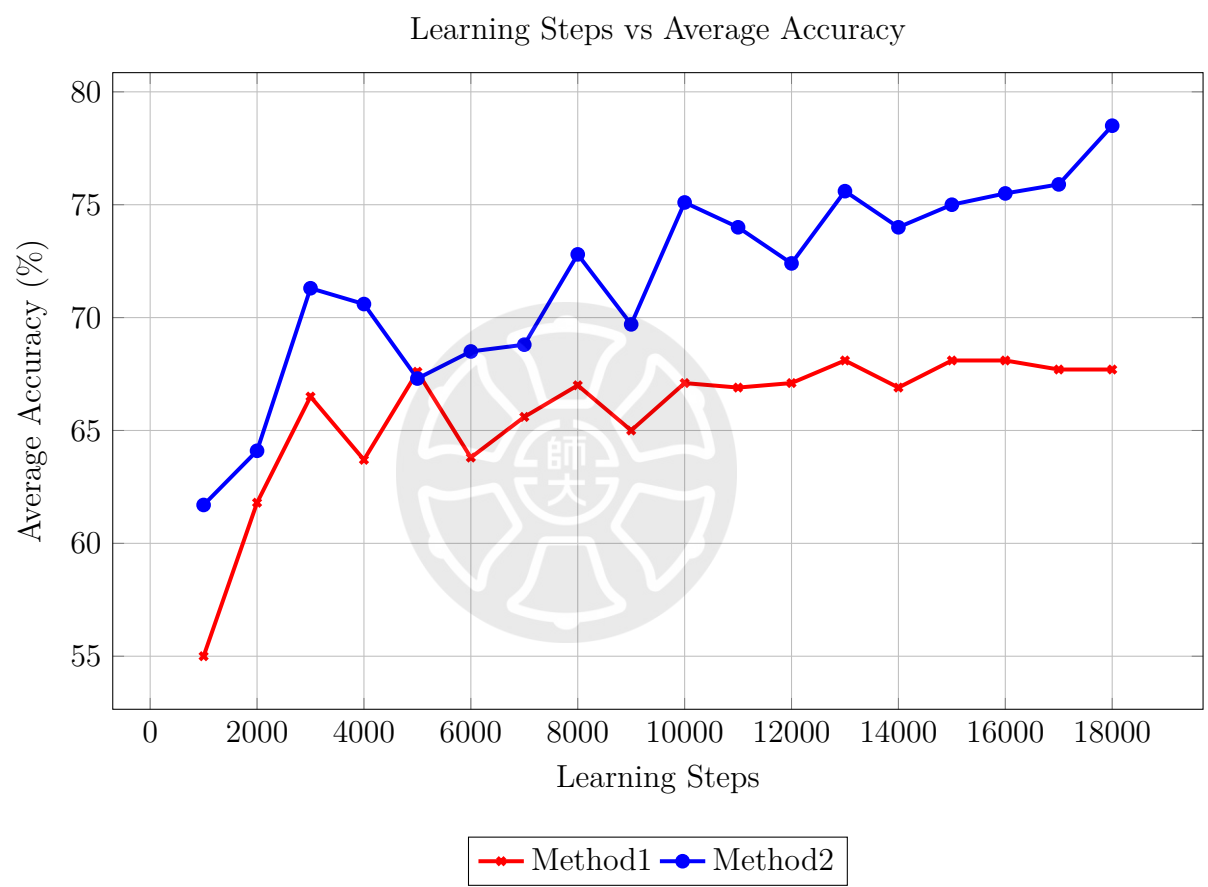


圖 4.1: 兩種斷詞方法的平均正確率

響。為了要更了解 3 種方式對於模型學習的影響，所以我們設計了 4 種不同的資料集，混合使用了 3 種技術。這四種資料集的配置為：「Original」是未經任何處理的資料集、「Function」是將乘法除法運算轉換成函數符號處理的資料集、「Function + Relevance-Control」是採用了函數運算以及將問題編排進解答的資料集、「Function + Relevance-Control + Sequence-Control」則是經過所有方法做調整之資料集。圖4.2呈現 4 種不同的模型在測試集中得到的平均正確率。其中縱軸是平均正確率，橫軸為 Learning steps，所以本張圖表示了 4 種資料集在不同的 Learning steps 下對本任務的表現。

首先，我們觀察到使用「Original」配置的資料集進行微調訓練的模型的平均準確率相當低。儘管經過微調後，平均準確率有小幅上升，但仍然相當低。這表明本任務對於 LM 來說相對困難。若沒有額外的設計，難以達成本任務設定之目標。

當 LM 使用了「Function」資料進行微調後，我們觀察到整體的平均準確性大幅上升，相較於原始配置提升了約 30%。從中可以推斷，在本任務中，LM 難以處理算術任務中的乘法和除法操作，因此「Function」的使用仍然是必要的。

接下來，我們觀察到「Function」與「Function + Relevance-Control」兩種資料集對於微調 LM 的影響。在初期，兩者對於 LM 在此任務上的表現相當，因為兩者在 Learning step 為 1000 時的準確率相近。然而，隨著微調的進行，「Function + Relevance-Control」配置的重要性逐漸凸顯出來。相較於「Function」配置，「Function + Relevance-Control」配置對於 LM 在處理此任務上提供了更多的幫助，因此在相同的 Learning steps 下可以提升更多的準確率。

最後，我們關注加入「Sequence-Control」編排技術的影響。然而，我們發現在平均準確性上，有無「Sequence-Control」並沒有太大的差距。在微調初期，加入「Sequence-Control」技術會導致更高的震盪，但到了微調後期，加入「Sequence-Control」的模型基本上可以稍微領先於未加入「Sequence-Control」的模型。

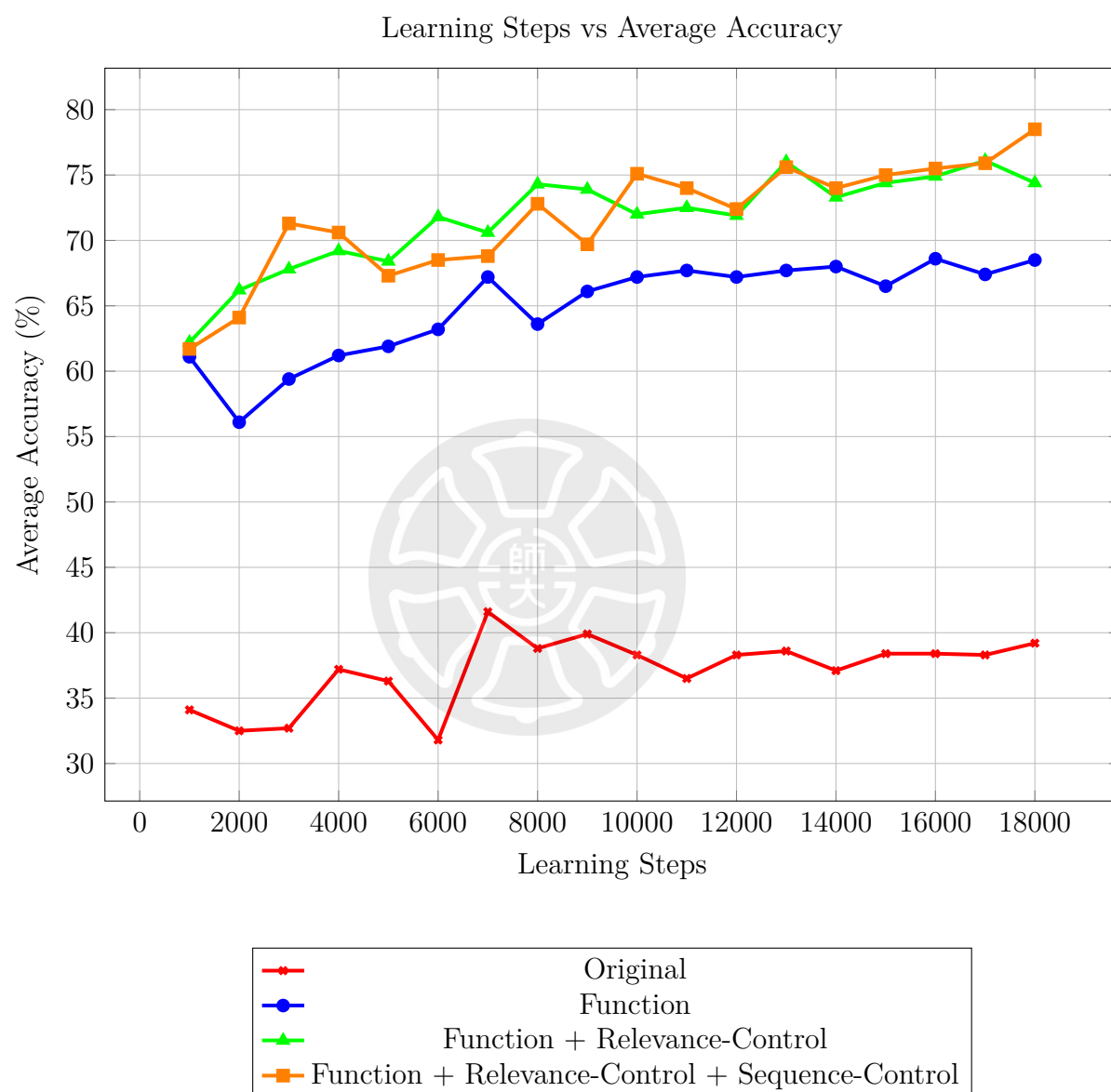


圖 4.2: 4 種不同資料集配置的平均正確率

接下來，我們將深入探討不同設計對此任務影響的具體細節。對於 LM 而言，乘法和除法常常被視為較為困難的運算任務。為了緩解這一挑戰，我們引入了「Function」功能，目的是讓模型避免直接執行這些困難的計算。加入「Function」後，模型的平均正確率顯著提高。然而，為了深入理解這一改進是否僅僅因為「Function」簡化了計算問題，或者是因為「Function」的使用導致問題的整體複雜度降低，進而影響了正確率的提升，我們對「Function」的使用頻率進行了統計分析。

圖4.3展示了使用「Function」與未使用時平均正確率的差異，這種差異通常在 25% 至 30% 之間。圖中的「Ratio1」表示在所有回答中使用「Function」的比例；「Ratio2」則表示在所有答案中既使用了「Function」且答案正確的比例。值得注意的是，儘管「Ratio1」的比例高出「Accuracy Difference」約 10% 至 15%，「Ratio2」的比例與「Accuracy Difference」卻大致相同。因此，我們推斷「Function」的正確使用是提升本任務的主要原因，而減少問題的複雜度。

然而，即使引入了 Function 功能，LM 仍然可能產生錯誤答案。為了更深入地探究這些錯誤是否源於 LM 在 Function 的應用錯誤或是其他因素，我們進行了錯誤答案的詳細統計並將錯誤分為兩類：

1. **Function 使用錯誤**：這類錯誤涵蓋了函數選擇不當及參數設置不正確的情況。當 LM 錯誤地選擇了函數或者輸入了錯誤的參數時，這些情況將被記錄為 Function 使用錯誤。
2. **其他錯誤**：除了函數使用錯誤外，所有其他類型的錯誤都被歸類在此類別。

這種分類方法幫助我們精確地理解 LM 在 Function 的實際運用情況。表4.1展示了對 195 個使用函數錯誤的案例進行的錯誤分類結果，從中我們觀察到因使用函數而產生的錯誤相對較少。儘管 LM 在使用函數時仍有錯誤出現，這一策略顯著降低了在執行乘法與除法計算時的錯誤率。

當我們引入 Relevance-Control 技術以後，LM 在解答任務的正確率顯著提升。

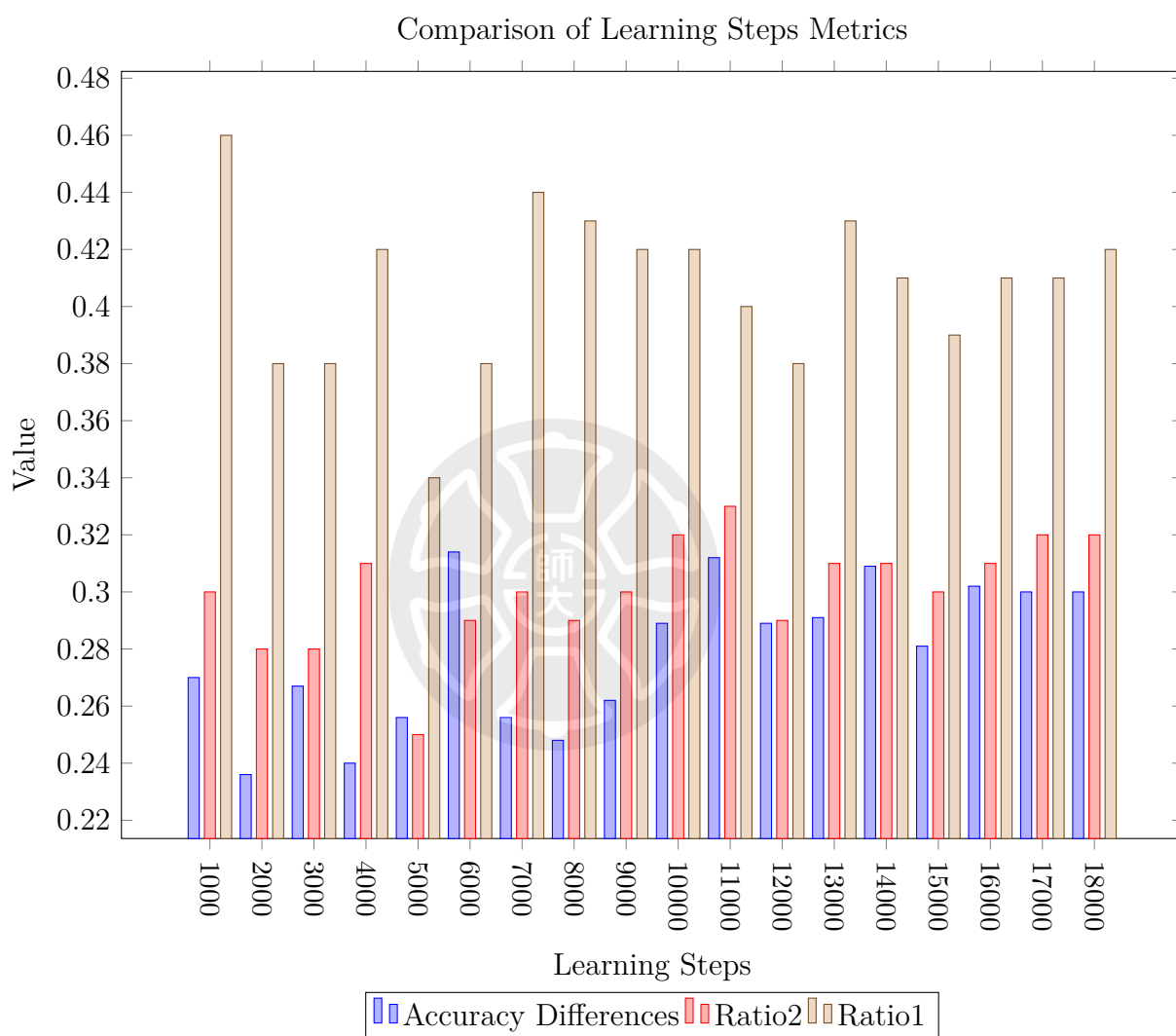


圖 4.3: Accuracy difference 與 Function 使用率與 Function 使用並正確之比較
 Accuracy difference 表示為「Oringal」與「Function」正確率之差;Ratio2 表示使用
 Function 並且正確的機率;Ratio1 表示使用 Function 的機率。

表 4.1: 在「Function」配置下，錯誤類型的數量統計

Type of wrong	Number of wrong
Function using wrong	86
Other type wrong	109

為深入探討 Relevance-Control 對研究結果的影響，我們對使用與未使用 Relevance-Control 的兩種情況進行了分析。先前研究表明，LM 在生成答案時會參考之前生成的答案，隨著越來越多答案的產生，模型可能會吸收到不必要的信息，從而影響計算的準確性。

為驗證上述推論，我們讓 LM 對所有問題生成 1 至 5 個不等的答案。所有答案按生成順序進行編號，最早生成的答案編號為 A，以此類推。我們統計了相同編號的答案數量及其正確率，以此分析答案順序對正確率的影響。然而大部分問題生成了三種答案，而生成四種或更多答案的頻率明顯較低。因此，我們只計算了前三個答案（A、B、C）的正確率。

在圖4.4中，橫軸代表答案的編號（A、B、C），縱軸表示相應編號答案的正確率。例如若存在 1000 個問題，且模型對每個問題生成了多種答案，我們便統計了相同編號下的答案中正確與錯誤的數量，並計算該編號的正確率。

較晚生成的答案受到先前答案的影響較大，從而導致正確率下降。使用 Relevance-Control 後，編號 B 和 C 的正確率明顯提升。這證明了 Relevance-Control 技術能有效減少由於先前答案影響而導致後續解答正確率下降的問題，此外同時也給予模型更多的特症去學習所以造成編號 A 正確率提升的現象。

最後，我們想要探討關於 Sequence-Control 的部分，在圖4.2中可以發現當我們加入 Sequence-Control 時卻沒有幫助平均正確率的上升，這與我們的預期有很大的差距，所以我們再次深入了解兩者有何種的不同之處。儘管兩者在平均正確率上幾近相同的表現，但在兩者的泛化能力卻有著不同的表現。

圖4.5呈現了在不同 Learning step 下，所有解答皆錯誤的機率。例如在 600 題的測

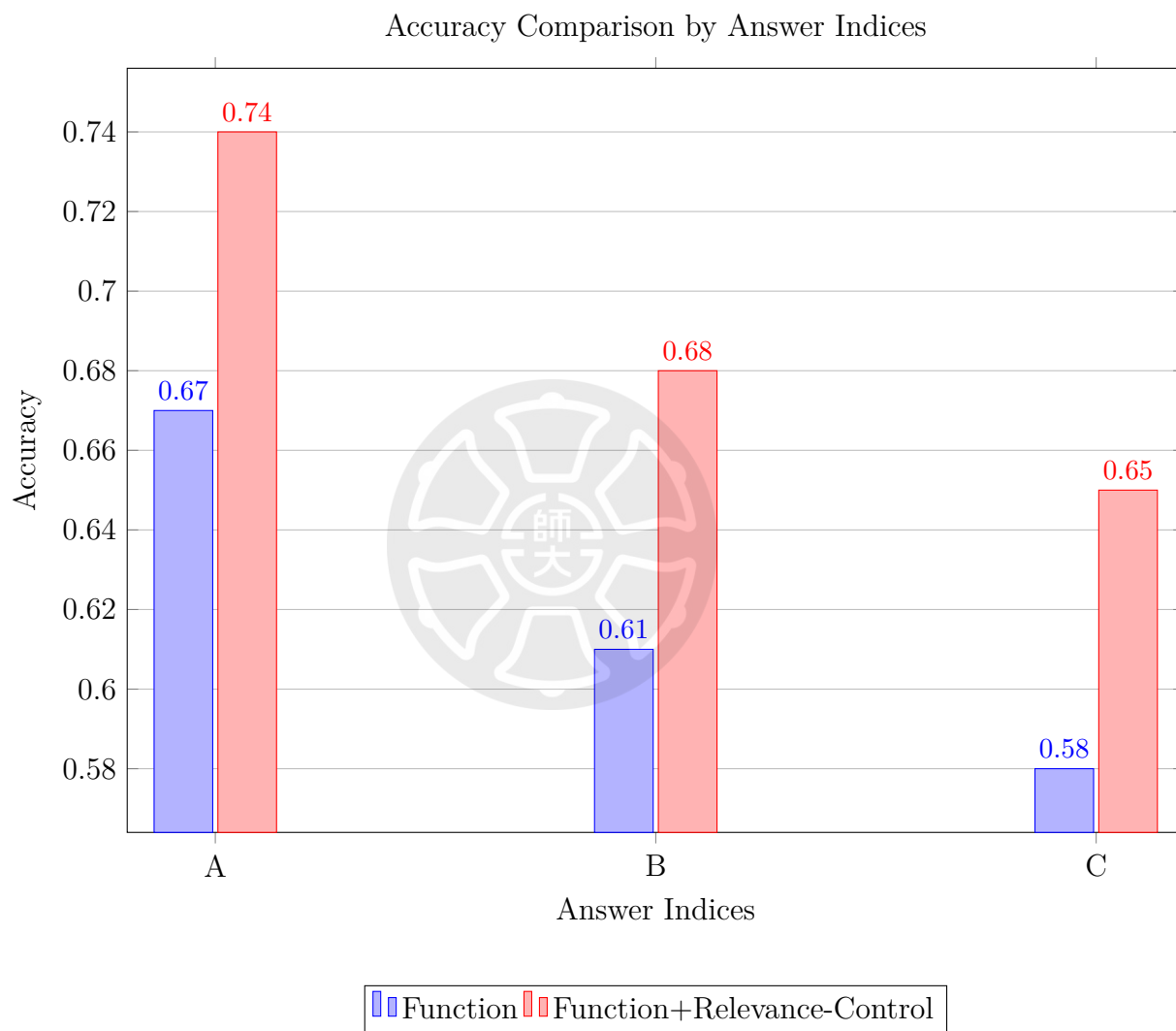


圖 4.4: 此圖展示模型在不同配置下對前三個回答 (A、B、C) 的準確率

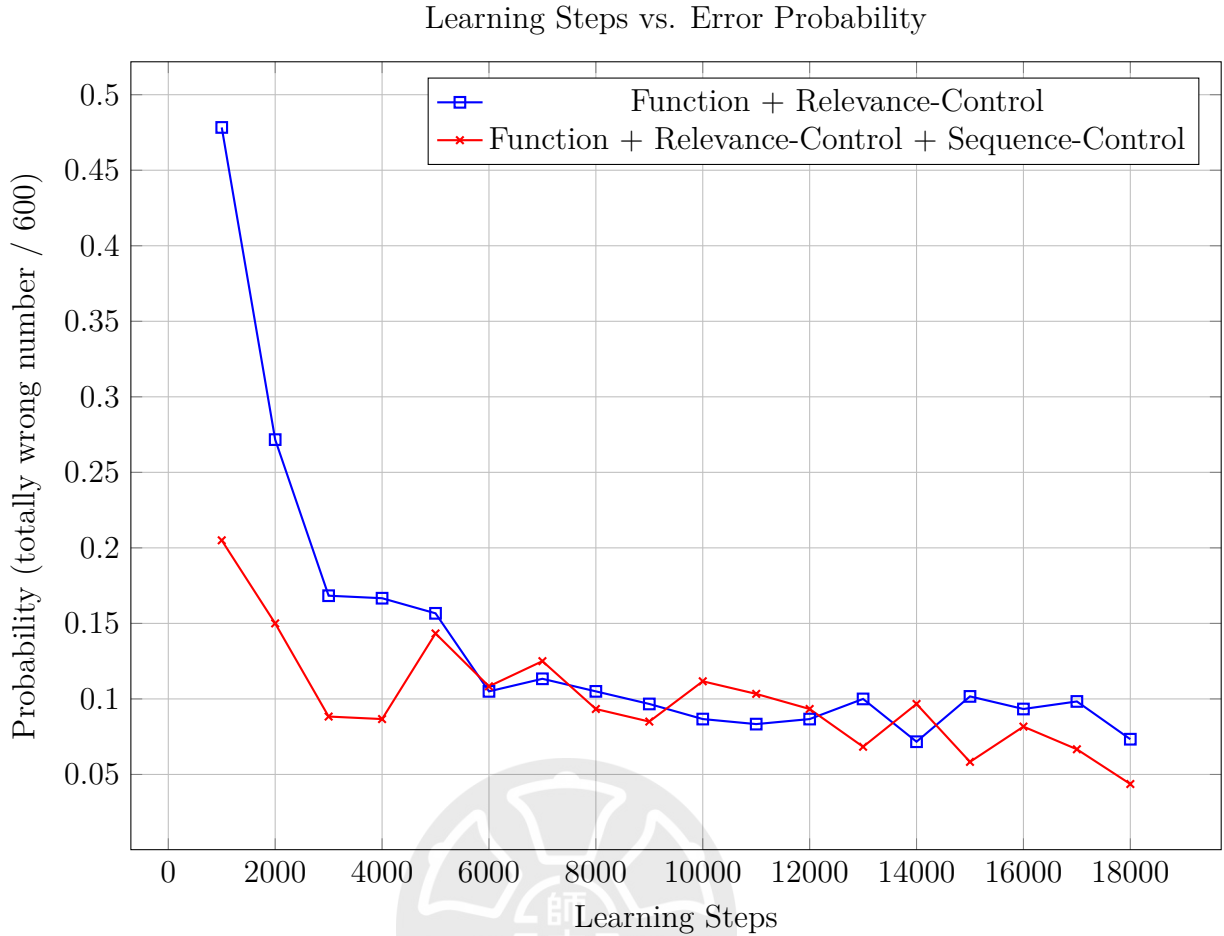


圖 4.5: 「Function+Relevance-Control」與「Function+Relevance-Control+Sequence-Control」的全部錯誤率，600 為測試題目的總數

試中共有 10 題的回答是全部錯誤的時，則 $10/600$ 為全部錯誤的機率。以及圖4.6呈現了在 Learning step = 18000 時，加入「Sequence-Control」與未加入時在不同 Numerical quantity 下全部錯誤的機率，可以發現在未加入 Sequence-Control 模型的錯誤率在 Quantity 較小時會明顯的高於加入 Sequence-Control 的模型，隨著 quantity 上升則兩者的錯誤率則越來越相似。

這種方法可以幫助我們更深入地了解 LM 在問題泛化上的能力。從結果可以看出，未加入 Sequence-Control 的模型在此評估中全錯的機率明顯高於加入 Sequence-Control 的模型。換言之，使用 Sequence-Control 可以有效減少 LM 在這項任務中全部答錯的情況。

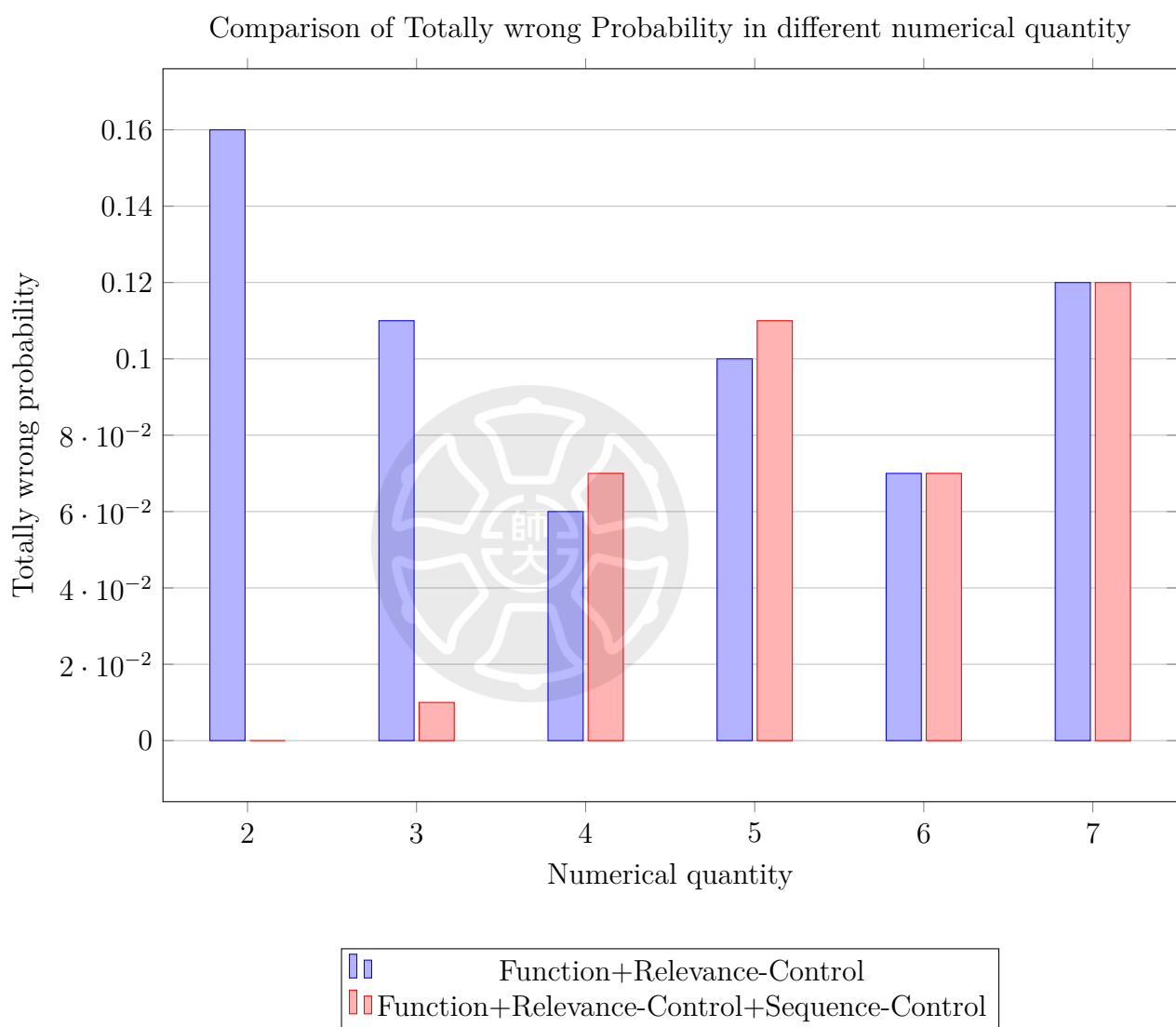


圖 4.6: 此圖展示有無 Sequence-Control 在不同數值數量下全部錯誤的機率

4.2 多樣性

上一節我們探討了經過微調的模型在數學算術任務上的正確性，本節將討論模型在生成多種解答的多樣性。我們將多樣性分為兩個主要部分：解答的獨特性以及豐富性。

4.2.1 獨特性

在自然語言生成領域，衡量兩段文本的獨特性通常涉及計算它們的相似度，這是因為即便是不同的文本也可能傳達相同的概念。然而，在本研究的數學計算上，只要數學表達式存在變化，我們便認為生成的答案已與原始問題有所不同。儘管在對 LM 進行微調時要求解答必須不同，但這並不保證解答的絕對獨特性。

為了量化生成任務的唯一性，我們計算了問題中所有解答的唯一率。例如，LM 對一問題生成了五個答案，其中兩個答案相同，則唯一率為 $4/5 = 0.8$ 。表4.2顯示了不同數據集下解答的唯一率，儘管存在一些重複現象，但在 90% 的情況下所有答案均不相同。在前一小節中所計算之平均正確率已將所有重複的內容視為錯誤選項。

表 4.2: 此表呈現不同 Dataset 的唯一率

Dataset	Uniqueness Rate
Function	0.92
Function+Relevance-Control	0.92
Function+Relevance-Control+Sequence-Control	0.93

4.2.2 豐富性

首先，我們對在 600 道題目中各個模型生成的解答進行了詳細的分析，並將結果整理在表4.3中。從這些數據中可以明顯觀察到，每個模型平均能夠提供 2 種以上的解答。具體來說，我們注意到「Function+Relevance-Control」模型的平均解

答數量相比僅使用「Function」模型有所下降。此外，當我們進一步在模型中加入「Sequence-Control」功能後，平均解答數量再次表現出上升的趨勢。

進一步的分析揭示了一個有趣的現象：相比於「Function」和「Function + Relevance-Control」配置，「Function + Relevance-Control + Sequence-Control」模型展現出較低的錯誤率。這一發現引起了我們的深入思考。我們認為，加入「Relevance-Control」功能後，模型在面對問題時能夠進行更深入的思考和重新評估，這可能有助於減少錯誤解答的生成，並提高解答的質量。

另一方面，當我們將「Sequence-Control」功能加入時，雖然整體解答的數量有下降趨勢，但模型生成的正確解答總數卻顯示出增長的趨勢。這表明「Sequence-Control」為模型提供了更多的結構化信息，使得模型有更多的信息判斷問題應該如何處理，進而減少錯誤解答的機率。

表 4.3: 此表呈現不同 Dataset 的平均解答數量

Dataset	Average answers	Average correct answers
Function	2.96	2.15
Function + Relevance-Control	2.84	1.98
Function + Relevance-Control + Sequence-Control	2.64	2.11

在本研究中，圖4.7展示了不同模型在處理具有不同數值數量的表達式時所產生的平均解答數量。這揭示了各模型在處理不同數值數量的問題時的表現。在我們的數據集中，只收集到了 2 到 5 數值數量的問題，在圖中可以看到每一種數值數量的平均解答數量。

而 3 種不同的模型在解答數量的表現趨勢相近，都是隨著數據集的設定所生成答案。但當數值數量超過數據集設定後，可以發現「Function」、「Function + Relevance-Control + Sequence-Control」兩者的模型在解答數量上仍是持續上升的現象。以及，可以發現儘管「Function + Relevance-Control」在大多問題中能夠提出較多的解答數量，但在表4.3正確解答平均數卻小於「Function + Relevance-Control +

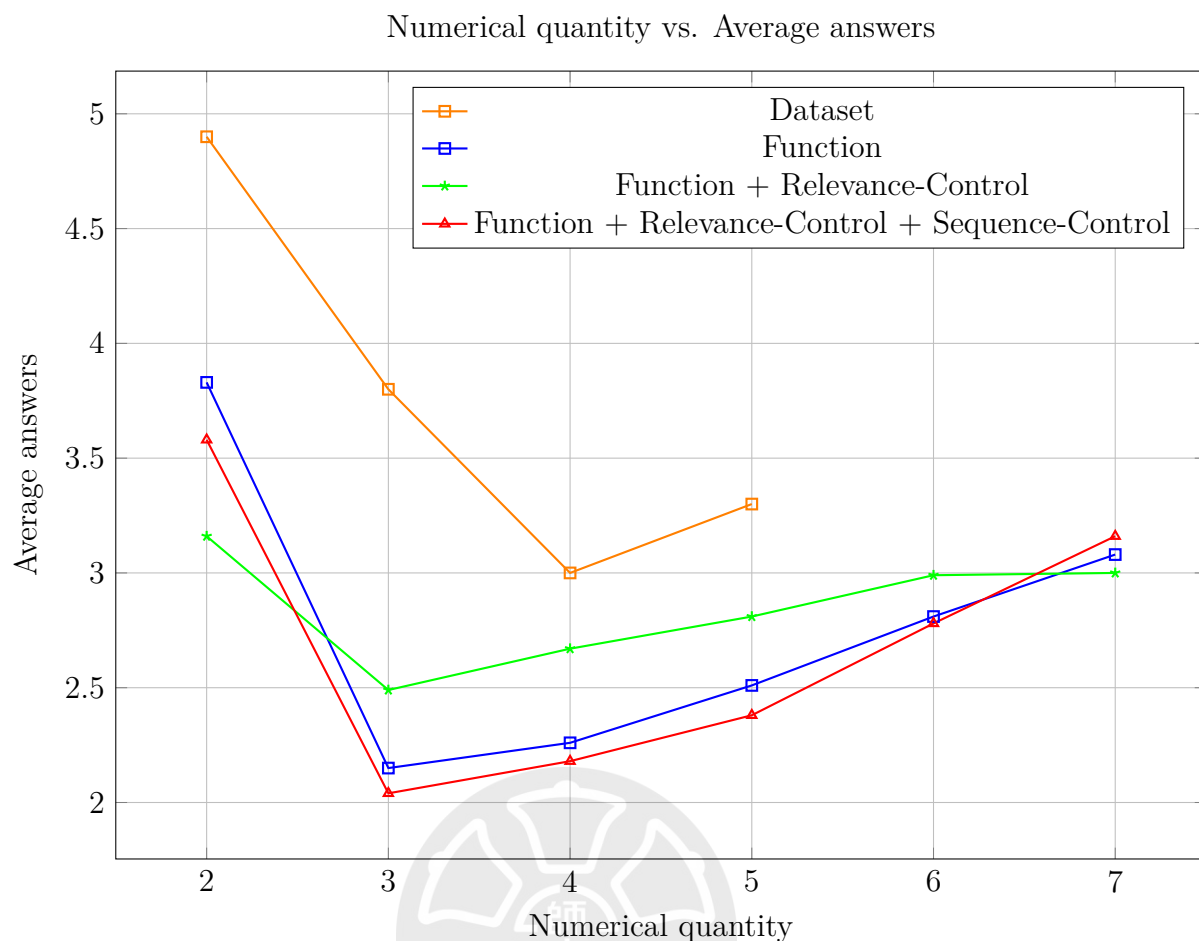


圖 4.7: 不同模型對於在不同數值數量下的平均解答數量

Sequence-Control」。

4.3 消融實驗 Ablation study

為了深入理解各設計方法對本研究任務的影響，我們進行了一系列剔除研究 (Ablation study)。考慮到函數在先前研究中對成果的顯著貢獻，本節將不對函數進行單獨分析。相反，我們將焦點放在關聯控制 (Relevance-Control) 和序列控制 (Sequence-Control) 兩個方面，並選擇了以下兩個資料集進行探討：「Function + Relevance-Control」和「Function + Sequence-Control」。

圖4.8展示了這兩種配置與原始模型在相同學習階段的平均準確率。從圖中可以觀察到，在僅使用「Function+Sequence-Control」的配置下，模型的平均準確率遠低於

「Function+Relevance-Control」以及「Function+Relevance-Control + Sequence-Control」的配置。單看正確率的部分單純使用「Relevance-Control」可以取得更好的效果。

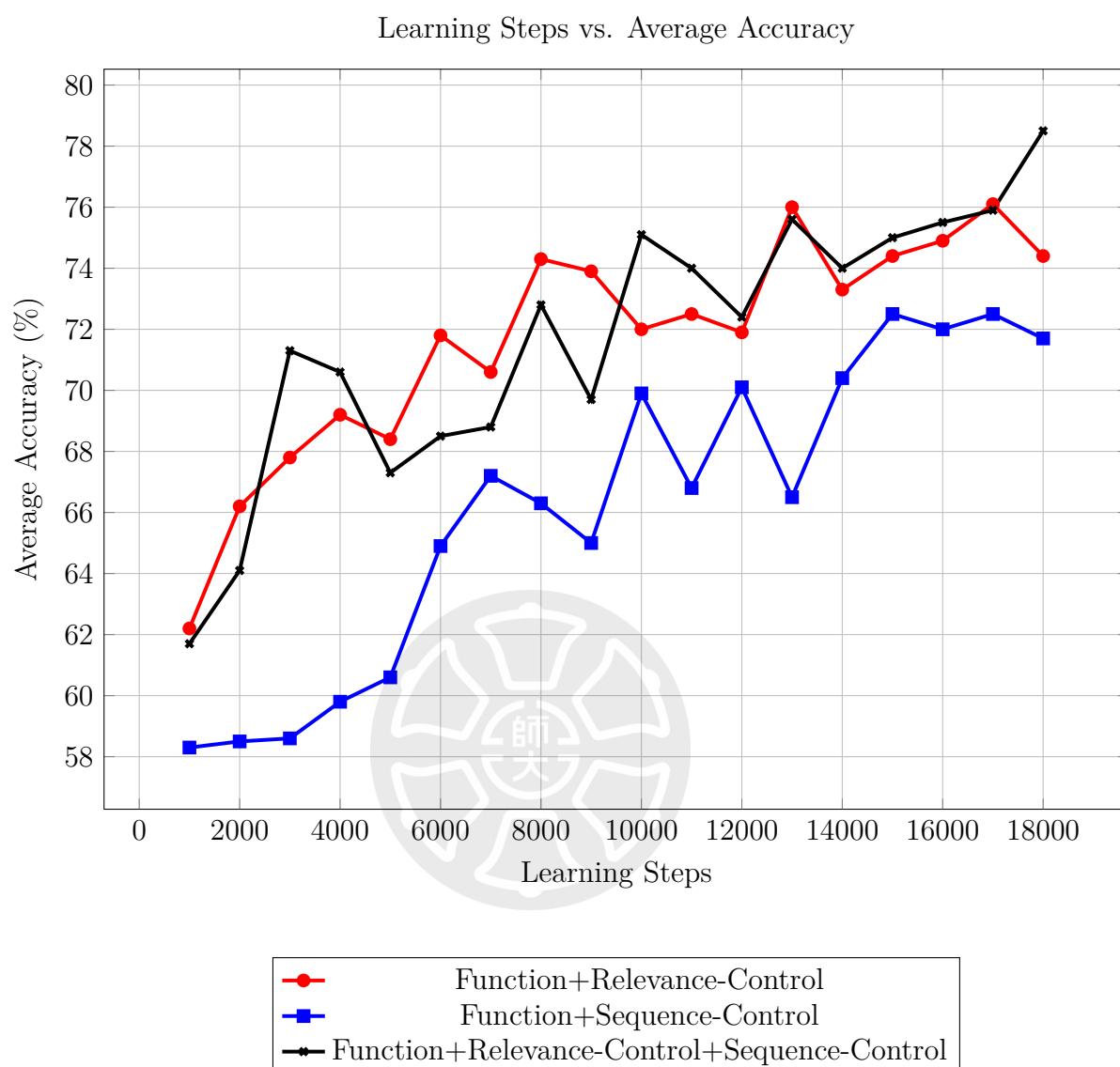


圖 4.8: 「Function+Relevance-Control」、「Function+Sequence-Control」、和「Function+Relevance-Control+Sequence-Control」平均正確率比較圖

為了深入理解記憶性 (Relevance-Control) 與順序性 (Sequence-Control) 對模型性能的具體影響，我們進行了一系列的分析，其結果如圖4.9所展示。該圖顯示了在不同解答編號下，使用「Function+Relevance-Control」與「Function+Sequence-Control」配置的模型的正確率對比。具體來看，在使用「Function+Sequence-Control」的配置中，模型在第一個解答的正確率顯著較高，然而隨著解答編號的增加，其正確率急劇下降，至第三個解答時正確率約為 0.53。這可能表明，雖然「Function+Sequence-Control」配

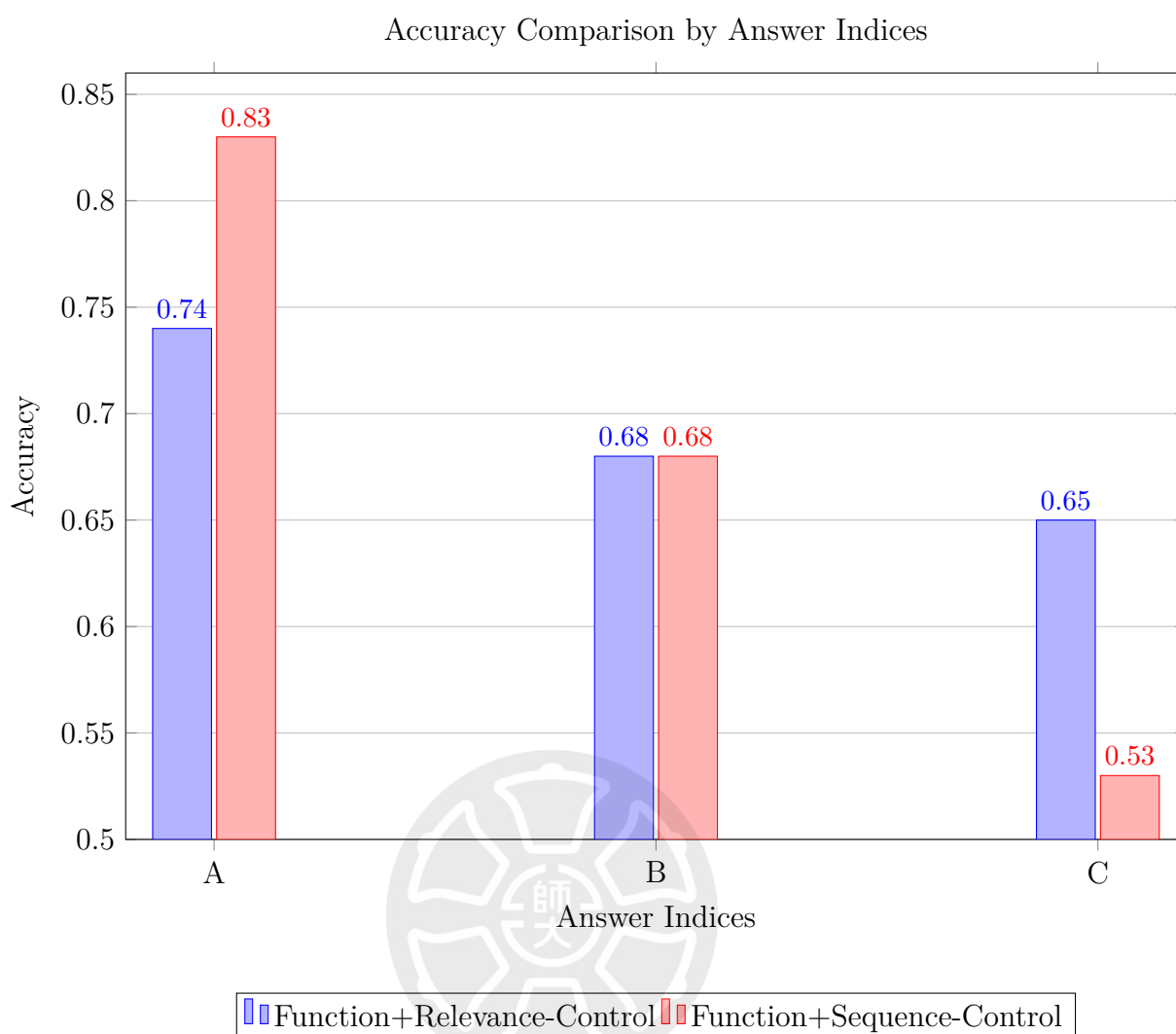


圖 4.9: 此柱狀圖展示了模型在不同配置下對首三個回答（A、B、C）的準確率

置能夠提升初期解答的精確度，其持續性和對複雜問題的適應能力卻受到限制。

相反，當使用了「Function+Relevance-Control」配置時，儘管整體正確率也隨著解答編號的增加而有所下降，這一下降的趨勢卻相對平緩。這表明「Relevance-Control」賦予了模型更佳的信息保留能力，這種特性助於模型在整個問題解答過程中保持較高的正確率。

綜上所述，「Relevance-Control」與「Sequence-Control」各自對模型正確率的影響有著顯著差異，這強調了在模型設計時考慮適當的功能增強對提升模型整體性能的重要性。

4.4 錯誤分析

在前面的章節中，我們對斷詞系統以及資料集設計對本任務的影響進行探討，可以發現 Function 對於本任務有相當大的幫助，可以減少在乘法以及除法上錯誤的發生;Relevance-Control 可以減少 LM 在推理期間參考到非必要資訊，所造成較晚生成解答的推理錯誤;Sequence-Control 儘管在正確率的部分並沒有太多的增加，但是卻可以減少 LM 在推理問題全部錯誤的情形。

在本任務中的問題具有無限多種可能性，以及用於微調之資料集並不能覆蓋所有的情況，而微調資料集中問題的數值數量介於 2 到 5 之間，共有 50000 筆資料。本節中所呈現的內容皆是使用「Function + Relevance-Control + Sequence-Control」配置的 LM，在我們的測試資料集中問題的數值數量為 2 到 7 之間，而每一種數值數量的問題都有 100 道題目進行測試，我們分別統計了不同數值數量下的正確率以及多樣性。

表4.4中呈現了在不同數值數量下的平均正確率，隨著數值數量的上升平均正確率也不斷地下降，但我們可以發現在數值數量為 7 時卻可以達到正確率 60% 以及在平均解答數量上也可以提出多於一個的正確解答，儘管在我們的訓練資料集中未曾出現過相同數值數量的問題。儘管隨著問題數值數量的上升會導致正確率下降，但隨著數值數量的增加，也可以得到與其他數值數量相差無幾的正確解答數量。

表 4.4: 不同數值數量下的平均正確率以及平均正確解答數量

Number Quantity	Average Accuracy	Average number of Correct answers
2	0.96	3.4
3	0.93	1.85
4	0.79	1.76
5	0.70	1.68
6	0.68	1.94
7	0.62	1.99

在數學問題中存在著多種的變化性，不管是數值上的變化或者是問題結構上的變化，都是造成本任務難以達成的原因之一。在我們訓練資料集中存在著多種不同的問

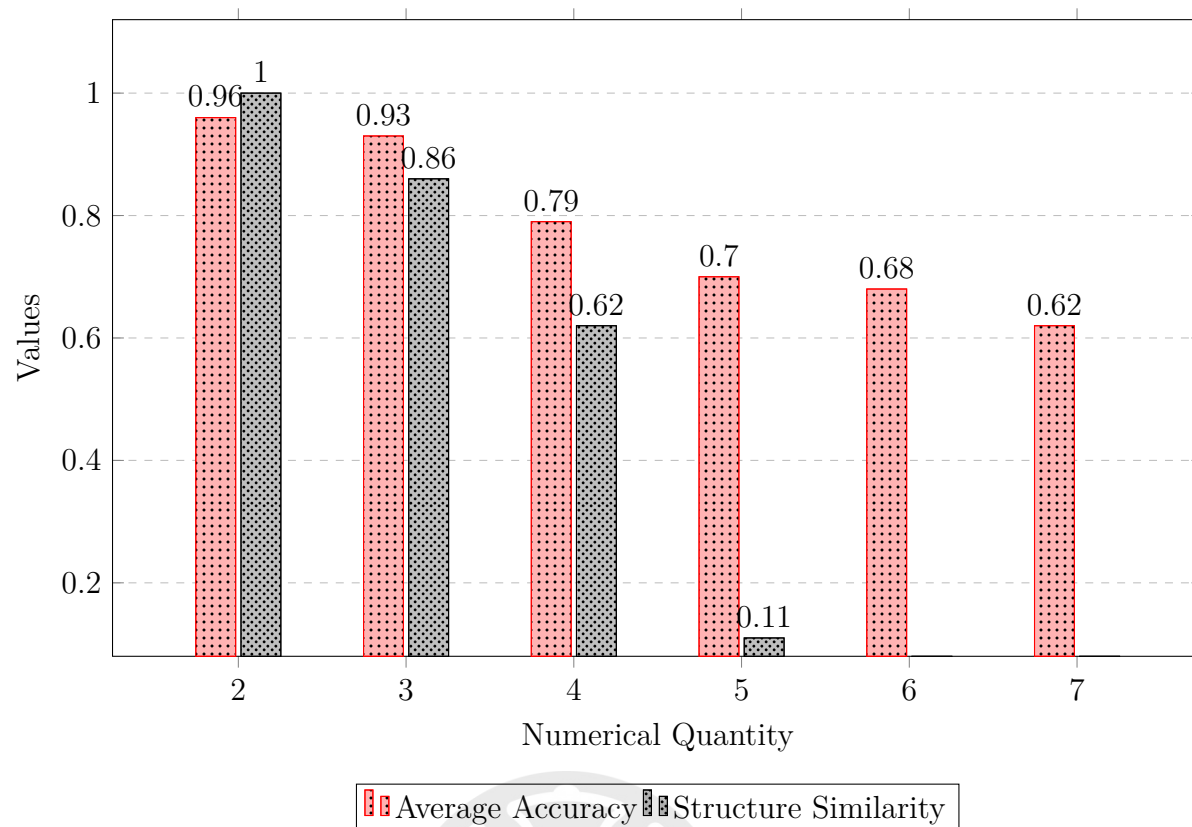


圖 4.10: 在不同數值數量下，正確率與結構相似率的比較

題結構，而用於測試問題中也有可能包含訓練資料集中出現過的結構，而我們想要 LM 是否能自行學習未曾出現過的數學問題結構。所以我們統計了在相同數值數量下訓練資料集問題結構出現在測試問題中的機率，如圖4.10所示橫軸為數值數量從 2 到 7 之間，以及縱軸為正確率以及相似結構的機率，可以發現除了數值數量為 2 的情況下，剩餘數值數量的平均正確率都高於相似結構的機率，在數值數量為 6 與 7 時更加的明顯可見。所以我們可以說 LM 可以透過學習過往的資料自行推理出未曾出現過的數學問題。

第五章 結論與未來研究

在過去的數學教育中，解題和講解任務主要由教師承擔。隨著大型語言模型（LLM）的發展，這些任務逐漸可以由 LLM 接管，儘管在解題過程中仍需教師的輔助。研究指出，提供學生多樣化的解題方法能顯著提升他們的學習動力及成效。因此，本研究通過對 LLM 進行監督式微調，使其能生成多種解答方式。我們提出了三種資料集設計方法，分析這些方法組合對 LLM 微調後表現的影響。實驗結果顯示，同時運用這三種方法能將平均正確率提升達 37%。此外，這些設計方法分別針對算術計算的難度、減少解答先後出現的影響以及對解答排列順序，對 LLM 的任務表現有顯著影響。

透過錯誤分析，我們證明了 LLM 能夠學習過往的內容，並推斷未曾見過的數學問題結構。這表明，儘管系統的準確性仍有提升空間，但多次嘗試後，正確答案的數量有明顯增加。本研究作為一個多元化數學解題系統的一部分，未來將探索如何使模型生成的解答更加人性化，提升解題系統的泛用性，並增強其在遇到新問題時的穩定性和單次推理的正確性。

目前，許多研究團隊正致力於提升 LLM 在數學算術任務上的表現。雖然我們的研究方向略有不同，我們希望未來能結合數據格式化技術，有效解決乘法及除法的挑戰。

我們計劃透過強化學習微調技術，使人工智慧模型不僅是解題工具，而是能根據每位學生的學習風格和能力進行個性化教學，如同量身訂做的家教老師。此外，我們也計劃整合影音生成系統，讓學生在提問時可以獲得包含視覺和聲音的詳細解說影片，從而提供一個更豐富的互動學習體驗。

除了對數算算數任務進行擴展以外，我們將構思如何擴展這一系統應用於解決數學教育中的表格問題和應用問題，這將進一步提升學生對數學概念的理解和應用能力，為學生的學習上提供更多多元化的解題方式。



附錄 A — Dataset

在符號表達中，我們將 n 位整數記作 nD ；以及利用 n 位整數部分和 m 位小數部分表示一個小數，記作 $nD.mD$ 。

A.1 Train dataset

表 A.1: 訓練資料集的問題符號表示。

Symbols	
Complexity 2	
$nD+-nD$	$nD*nD$
$nD*(+-nD)$	nD/nD
$\frac{nD}{nD}$	$nD+-(-nD)$
$+-nD+-nD$	$(nD+-nD)$
(nD/nD)	$(nD*nD)$
Complexity 3	
$nD+-nD+-nD$	$nD+-nD*nD$
$nD+-nD/nD$	$nD+-\frac{nD}{nD}$
$nD*\frac{nD}{nD}$	$nD*nD*nD$
Continued on next page	

表 A.1 continued from previous page

Symbols	
$nD+-nD^*(+-nD)$	$+ -nD+-nD^*nD$
$nD/nD/nD$	$\frac{(nD+-nD)}{nD}$
nD^*nD/nD	$nD+- (nD+-nD)$
$nD+- (nD/nD)$	$nD/(nD+-nD)$
$(nD/nD)^*nD$	$nD^*nD+-nD$
$(nD+-nD)+-nD$	$(nD+-nD)/nD$
$nD^*(nD+-nD)$	$nD/nD+-nD$
$(nD+-nD)^*nD$	$nD+- (nD^*nD)$
$(nD+-nD+-nD)$	$\frac{nD.mD}{nD}$
$(nD+- (nD+-nD))$	nD/nD^*nD
$nD/(nD^*nD)$	
Complexity 4	
$nD+-nD^*(nD+-nD)$	$nD+-nD+-nD^*nD$
$nD^*((nD+-nD)+-nD)$	$nD^*(nD+- (nD+-nD))$
$nD^*(nD+-nD+-nD)$	$nD+-nD^*nD/nD$
$\frac{nD}{nD} + -\frac{nD}{nD}$	$nD+-nD^*(nD)/nD$
$nD+-nD/nD^*nD$	$nD+- (nD+-nD^*nD)$
$nD+- (nD^*nD+-nD)$	$nD+- (nD^*(nD+-nD))$
$+ -nD+-nD+-nD^*nD$	$nD+-nD/nD/nD$
$(nD+-nD)+-nD+-nD$	$nD+- (nD+-nD)+-nD$
$\frac{(nD+-nD)}{(nD+-nD)}$	$nD+-nD+-nD+-nD$
$nD+-nD/nD+-nD$	$nD/nD+-nD^*nD$

Continued on next page

表 A.1 continued from previous page

Symbols	
$nD/nD/nD/nD$	$nD^*nD/nD/nD$
$nD+-nD+-nD/nD$	$nD+-nD^*nD^*nD$
$nD/nD+-nD/nD$	$nD^*nD+-(nD^*nD)$
$nD/nD/nD+-nD$	nD/nD^*nD^*nD
$nD^*nD^*nD+-nD$	nD^*nD^*nD/nD
$(nD^*nD)/nD/nD$	$(nD/nD+-nD)/nD$
$nD^*nD/nD+-nD$	$nD^*nD+-nD/nD$
$nD/nD^*nD/nD$	$(nD/nD/nD+-nD)$
$nD+-nD+-(nD^*nD)$	$nD+-(nD^*nD/nD)$
$(nD+-nD+-nD)+-nD$	$nD/nD^*(nD+-nD)$
$nD^*(nD+-nD)+-nD$	$nD^*nD^*nD^*nD$
$nD^*nD^*(nD/nD)$	$nD^*nD+-nD^*nD$
$(nD+-nD+-nD)^*nD$	$nD+-nD^*nD+-nD$
$nD/nD+-nD+-nD$	$nD^*nD+-(nD/nD)$
$nD+-((nD+-nD)+-nD)$	$\frac{nD.mD}{nD.mD}$
$nD+-(nD+-nD)^*nD$	$nD^*(nD+-nD)^*nD$
$(nD+-nD)/nD+-nD$	$nD+-\frac{nD.mD}{nD}$
$nD+-\frac{nD}{nD.mD}$	$nD^*(nD^*nD+-nD)$
$nD^*(nD/nD+-nD)$	$(nD+-nD)/(nD+-nD)$
$nD/nD^*nD+-nD$	$nD/nD/(nD+-nD)$
$nD+-nD+-\frac{nD}{nD}$	$nD+-((nD+-nD)^*nD)$
$nD+-\frac{(nD+-nD)}{nD}$	$(nD+-nD)^*nD+-nD$

Continued on next page

表 A.1 continued from previous page

Symbols	
$nD/nD/(nD/nD)$	$(nD/nD)/nD/nD$
$nD/(nD/nD)/nD$	$nD/(nD+-nD)+-nD$
$(nD*nD+-nD)/nD$	$nD*nD*(nD+-nD)$
Complexity 5	
$nD+-nD*nD*nD/nD$	$nD+-nD*nD/nD/nD$
$nD+-nD+-nD*nD*nD$	$nD+-nD+-\frac{nD}{nD}/nD$
$nD+-(+ -nD+-\frac{nD}{nD})/nD$	$nD+- (nD+-\frac{nD}{nD})/nD$
$nD+- (nD+-\frac{nD}{nD})*nD$	$nD+-nD+-\frac{nD}{nD}*nD$
$nD+-nD*nD+-nD*nD$	$nD+-nD.mD/nD.mD$
$(nD+-nD.mD)/nD.mD$	$nD+-\frac{nD.mD}{nD.mD}$
$nD*\frac{nD}{nD}+-\frac{nD}{nD}$	$nD*(nD+-nD*nD)+-nD$
$nD*(nD+-nD/nD)+-nD$	$nD*nD+-nD*nD+-nD$
$nD*(nD+-nD)*nD+-nD$	$nD/(nD+-nD+-nD)+-nD$
$nD+-nD+-nD+-nD+-nD$	$nD*nD+-nD+-nD*nD$
$nD*nD*nD+-nD+-nD$	$nD/nD*nD*nD+-nD$
$nD/nD*nD+-nD*nD$	$(nD+-nD)/(nD+-nD)+-nD$
$nD*nD*(nD+-nD)+-nD$	

A.2 Test dataset

表 A.2: 測試資料集的問題符號表示

Symbols	
Complexity 2	
(nD/nD)	$(nD*nD)$
nD/nD	$nD*nD$
$nD+-nD$	$(nD+-nD)$
Complexity 3	
$nD+-nD/nD$	$nD+-(nD+-nD)$
$nD+-(nD/nD)$	$nD/nD/nD$
$nD+-nD+-nD$	$nD*nD+-nD$
$nD+-nD*nD$	$nD/nD+-nD$
$nD*(nD*nD)$	$nD*(nD+-nD)$
$(nD+-nD+-nD)$	$nD*nD*nD$
$(nD+-nD)+-nD$	$nD+-(nD*nD)$
$(nD+-nD)*nD$	$nD/(nD*nD)$
$nD/(nD+-nD)$	$(nD/nD)+-nD$
$(nD*nD)*nD$	$(nD+-nD)/nD$
$nD*nD/nD$	$(nD*nD/nD)$
$(nD+-nD/nD)$	$nD/nD*nD$
$(nD*nD*nD)$	
Complexity 4	
$nD/(nD+-nD)/nD$	$(nD+-nD)/nD+-nD$
$nD+-nD+-nD+-nD$	$nD*nD*nD*nD$
$nD/nD+-(nD/nD)$	$nD+-nD/nD/nD$

Continued on next page

表 A.2 continued from previous page

Symbols	
$nD*nD+-nD+-nD$	$nD+-nD+-nD*nD$
$nD*(nD+-nD)*nD$	$nD+-nD*nD+-nD$
$nD*nD+-nD/nD$	$nD+-nD*(nD+-nD)$
$nD+-(nD*nD+-nD)$	$nD*nD+-(nD+-nD)$
$nD/nD*nD/nD$	$nD/nD+-nD+-nD$
$(nD+-nD/nD)+-nD$	$nD*nD+-nD*nD$
$(nD+-nD)+-nD+-nD$	$nD+-(nD/nD)+-nD$
$nD+-nD/nD+-nD$	$nD/(nD*nD)/nD$
$nD*nD/nD*nD$	$nD+-nD+-nD/nD$
$nD/(nD+-nD+-nD)$	$nD/nD/nD/nD$
$nD/(nD/nD)+-nD$	$nD*nD*nD/nD$
$nD/nD/(nD*nD)$	$(nD+-nD*nD)+-nD$
$nD*(nD*nD+-nD)$	$nD*(nD+-nD+-nD)$
$nD/nD*nD+-nD$	$nD/nD+-(nD+-nD)$
$nD+-nD+-(nD*nD)$	$nD+-nD*nD/nD$
$nD*nD/nD+-nD$	$nD/nD+-nD/nD$
$(nD+-nD/nD*nD)$	$nD/nD*nD*nD$
$nD+-nD/nD*nD$	$nD/nD/(nD+-nD)$
$nD/nD+-(nD*nD)$	$(nD*nD/nD)/nD$
$nD+-nD/(nD+-nD)$	$(nD+-nD+-nD*nD)$
$(nD+-nD)/nD/nD$	$nD+-nD+-(nD/nD)$
$nD/nD+-nD*nD$	$(nD*nD)+-nD+-nD$

Continued on next page

表 A.2 continued from previous page

Symbols	
$nD^*(nD^*nD)+-nD$	$nD+-(nD/nD/nD)$
$nD^*nD^*(nD+-nD)$	$nD+-(nD+-nD+-nD)$
Complexity 5	
$nD/nD^*nD+-nD/nD$	$nD/nD+-nD^*nD^*nD$
$nD^*nD^*nD^*nD/nD$	$nD^*nD/nD^*nD/nD$
$nD+-(nD^*nD)/nD+-nD$	$nD+-nD+-nD^*nD^*nD$
$nD/nD/nD+-nD/nD$	$nD+-nD/nD/nD^*nD$
$nD^*nD/nD+-nD+-nD$	$nD/nD^*nD/nD+-nD$
$nD+-nD^*nD^*nD+-nD$	$nD^*nD+-nD+-nD+-nD$
$nD/nD^*nD+-(nD+-nD)$	$nD+-nD^*nD+-nD^*nD$
$nD/nD^*nD+-nD+-nD$	$nD/nD+-nD^*nD+-nD$
$nD+-nD^*nD+-nD+-nD$	$nD/nD/nD+-nD+-nD$
$nD/nD+-nD^*nD/nD$	$nD+-nD^*nD+-nD/nD$
$nD+-nD+-nD+-nD^*nD$	$nD/nD+-nD+-nD+-nD$
$nD^*nD^*nD/nD+-nD$	$nD+-nD^*nD/nD+-nD$
$nD+-nD/nD+-nD+-nD$	$nD^*nD+-nD^*nD+-nD$
$nD+-nD^*(nD^*nD+-nD)$	$(nD/nD)+-nD^*nD/nD$
$nD^*nD^*nD+-(nD+-nD)$	$nD+-nD^*nD/(nD/nD)$
$nD^*nD+-nD/nD+-nD$	$nD+-nD/nD/nD/nD$
$nD/nD^*nD^*nD+-nD$	$nD^*(nD+-nD)/nD/nD$
$nD/(nD/nD/nD^*nD)$	$nD+-nD^*nD^*(nD+-nD)$
$nD+-nD+-(nD+-nD+-nD)$	$(nD/nD^*nD/nD^*nD)$

Continued on next page

表 A.2 continued from previous page

Symbols	
$nD+- (nD/nD)/nD+-nD$	$nD+-nD/nD+-nD/nD$
$nD/nD/nD*(nD+-nD)$	$nD*nD+-nD+-nD/nD$
$nD+-nD/(nD+-nD)+-nD$	$nD+-nD/nD/(nD+-nD)$
$nD/nD*nD+-nD*nD$	$nD/nD/nD/(nD/nD)$
$(nD+-nD+-nD+-nD*nD)$	$nD+-nD+-nD+-nD+-nD$
$nD+-nD/nD/nD+-nD$	$(nD*nD+-nD*nD)/nD$
$nD+-nD+-nD/(nD/nD)$	$nD/nD/nD*nD+-nD$
$nD+-nD+- (nD+-nD)*nD$	$nD/nD/nD/nD/nD$
$nD*nD+- (nD*nD)*nD$	$nD/nD/nD+-nD*nD$
$nD+-nD*nD/nD/nD$	$nD+-nD+-nD+-nD/nD$
$nD*nD*nD/nD*nD$	$nD*nD/nD/nD*nD$
$nD+-nD+- (nD*nD*nD)$	$nD/nD*nD/nD*nD$
$nD+-nD+-nD+- (nD+-nD)$	$nD*nD*nD+-nD+-nD$
$nD+-nD/nD*nD+-nD$	$nD*nD/nD+-nD*nD$
$nD*nD+-nD/nD/nD$	$nD+-nD+-nD*nD+-nD$
$nD+-nD+- (nD+-nD)/nD$	$nD+-nD+-nD*nD/nD$
$nD+- (nD/nD)*nD/nD$	
Complexity 6	
$nD+-nD+-nD+-nD+-nD*nD$	$nD/nD/nD+-nD/nD+-nD$
$nD+-nD/nD*nD+-nD+-nD$	$nD+-nD*nD*nD*nD/nD$
$nD/nD+-nD*nD+- (nD+-nD)$	$(nD*nD+-nD/nD*nD)+-nD$
$nD*nD/nD+-nD+-nD+-nD$	$nD/nD+-nD/nD+-nD+-nD$

Continued on next page

表 A.2 continued from previous page

Symbols	
$nD+-(nD*nD+-nD+-nD)+-nD$	$nD+-nD/nD+-nD*nD*nD$
$nD/nD+-nD*nD+-nD+-nD$	$(nD+-nD*nD+-nD+-nD/nD)$
$nD+-nD+-nD/(nD+-nD+-nD)$	$nD*nD/nD/nD/nD+-nD$
$nD*nD+-nD/nD+-nD+-nD$	$nD*nD*nD*nD*nD*nD$
$nD*nD*nD+-nD+-nD+-nD$	$nD*nD/nD/nD/nD*nD$
$nD+-nD/nD/(nD/nD)/nD$	$nD+-nD*nD+-nD+-nD/nD$
$nD/nD/nD*nD/nD/nD$	$nD/nD*nD/(nD+-nD+-nD)$
$nD/nD/nD/nD+-nD/nD$	$nD+-nD/nD+-nD+-nD*nD$
$nD+-nD*nD/nD+-nD+-nD$	$nD*nD+-nD*nD+-nD*nD$
$nD+-nD+-nD*nD+-nD/nD$	$nD+-nD/nD*nD+-nD*nD$
$nD*nD/nD*nD/nD*nD$	$nD+-nD+-nD+-nD*nD$
$(nD/nD*nD+-nD+-nD+-nD)$	$nD+-nD*nD*nD+-nD+-nD$
$(nD*nD)/nD+-nD/nD+-nD$	$nD*nD/(nD*nD/nD)+-nD$
$nD+-nD+-nD/nD*nD+-nD$	$nD*nD*nD+-nD*nD+-nD$
$nD+-nD*nD+-nD+-nD+-nD$	$(nD+-nD+-nD+-nD+-nD*nD)$
$nD/nD*nD*nD/nD+-nD$	$nD*nD/nD+-nD*nD+-nD$
$nD/nD*nD/nD*nD+-nD$	$nD/nD/nD+-nD+-nD/nD$
$nD*nD*nD*nD*nD+-nD$	$nD*(nD*nD+-nD/nD)+-nD$
$nD/nD/nD+-nD+-nD+-nD$	$nD+-nD/nD*nD/nD+-nD$
$nD+-nD+-nD+-nD+-nD/nD$	$nD/nD*nD*nD+-nD+-nD$
$nD+-nD*(nD+-nD)+-nD+-nD$	$nD*nD/nD+-nD/nD)+-nD$
$nD*nD+-nD+-nD+-nD*nD$	$nD*nD*nD+-nD*nD/nD$

Continued on next page

表 A.2 continued from previous page

Symbols	
$nD^*nD+-nD+-nD^*(nD+-nD)$	$(nD/nD+-nD+-nD+-nD)+-nD$
$nD/nD+-nD+-nD^*nD^*nD$	$nD^*nD/nD+-nD/nD^*nD$
$nD^*nD+-(nD/nD^*nD^*nD)$	$nD/nD+-nD+-nD+-nD+-nD$
$nD^*(nD/nD+-nD+-nD)+-nD$	$nD+-nD+-nD/nD^*nD^*nD$
$nD+-nD+-nD^*nD+-(nD^*nD)$	$nD/nD/nD^*nD/nD+-nD$
$(nD^*nD+-nD^*nD/nD^*nD)$	$nD+-nD+-nD^*(nD^*nD/nD)$
$nD+-nD+-nD+-nD+-nD+-nD$	$nD^*nD+-nD+-nD+-nD+-nD$
$nD+-nD+-nD+-nD/nD+-nD$	$nD^*(nD+-nD^*nD+-nD^*nD)$
$nD+-nD+-nD^*nD+-nD+-nD$	$nD+-nD+-nD+-nD/nD/nD$
$nD+-nD^*nD+-nD/nD+-nD$	$nD+-nD/nD+-nD+-(nD^*nD)$
$nD+-nD/nD+-nD+-nD+-nD$	$nD^*nD+-nD/nD^*nD+-nD$
$nD/(nD/nD+-nD)^*nD/nD$	$nD/nD^*nD+-nD/nD/nD$
$nD^*(nD+-nD+-nD+-nD+-nD)$	$nD/nD+-nD+-nD^*nD/nD$
$(nD^*nD^*nD+-nD^*nD)/nD$	$nD+-nD+-nD/(nD+-nD^*nD)$
$nD/nD+-nD+-nD/nD+-nD$	

Complexity 7

$nD+-nD^*nD+-nD^*nD+-nD^*nD$	$nD/nD^*nD^*nD+-nD/nD+-nD$
$nD+-nD^*nD+-nD+-nD+-nD^*nD$	$nD+-nD+-nD+-nD+-(nD+-nD/nD)$
$nD+-nD+-nD+-nD+-nD^*nD+-nD$	$nD+-nD+-nD/nD/nD+-nD+-nD$
$nD+-nD+-nD/nD/nD/nD+-nD$	$nD+-nD+-nD/nD+-nD+-nD/nD$
$nD^*nD+-nD+-nD+-nD/nD+-nD$	$(nD+-nD/nD+-nD+-nD/nD^*nD)$
$nD+-nD+-nD/nD^*nD+-nD+-nD$	$nD+-nD+-nD^*nD+-nD+-nD^*nD$

Continued on next page

表 A.2 continued from previous page

Symbols	
$nD+-nD^*(nD+-nD+-nD^*nD+-nD)$	$nD+-nD/nD/nD/nD^*nD^*nD$
$nD^*nD+-nD+-nD+-nD+-nD+-nD+-nD$	$nD/nD+-nD/nD+-nD+-nD+-nD$
$nD^*nD^*nD+-nD/nD/nD+-nD$	$nD+-nD+-nD/(nD^*nD+-nD)^*nD$
$nD/nD/nD/nD+-nD/nD^*nD$	$nD^*nD+-nD+-nD/nD/nD^*nD$
$nD^*nD+-nD+-nD/nD+-nD+-nD$	$nD/nD+-nD/nD+-nD+-nD^*nD$
$(nD/nD+-nD/nD+-nD/nD/nD)$	$nD^*nD+-nD^*nD/nD^*nD+-nD$
$(nD^*nD/nD/nD^*nD^*nD)^*nD$	$nD/nD/nD^*nD^*nD+-nD^*nD$
$nD+-nD+-nD+-nD/nD^*nD^*nD$	$nD+-nD/nD+-nD^*nD^*nD^*nD$
$nD+-nD+-nD^*nD/nD+-nD^*nD$	$nD/nD+-nD^*nD/nD/nD/nD$
$nD^*nD/(nD/nD)/nD+-nD+-nD$	$nD+-nD^*nD^*nD^*nD+-nD+-nD$
$nD+-nD+-nD/nD+-nD)+-nD/nD$	$nD^*nD+-nD+-nD^*nD+-nD+-nD$
$nD^*nD/nD^*nD+-nD+-nD/nD$	$nD+-nD/nD+-nD^*nD/nD+-nD$
$nD/nD^*nD+-nD+-nD^*nD^*nD$	$nD+-nD/nD/nD+-nD/nD+-nD$
$nD/nD^*nD^*nD^*nD+-nD+-nD$	$nD+-nD+-nD/nD^*(nD/nD)+-nD$
$nD+-nD/nD^*nD+-nD+-nD/nD$	$nD^*nD+-nD^*nD^*nD^*nD+-nD$
$nD+-nD+-nD+-nD+-nD+-nD+-nD$	$nD/nD+-nD^*nD+-nD+-nD+-nD$
$nD^*nD/nD/nD^*(nD+-nD^*nD)$	$nD+-nD/nD+-nD+-nD+-nD+-nD$
$nD+-nD+-nD+-nD/(nD/nD+-nD)$	$nD^*nD^*nD^*nD/(nD+-nD+-nD)$
$nD+-nD^*nD+-nD/nD+-nD/nD$	$nD+-nD+-nD/nD/nD/nD^*nD+-nD$
$nD+-nD^*nD^*nD+-nD+-nD^*nD$	$nD+-nD+-nD+-nD/nD)+-nD^*nD$
$nD+-nD^*(nD+-nD)+-nD^*nD/nD$	$(nD+-nD^*nD+-nD)^*nD+-nD^*nD$
$nD+-nD^*nD/nD/nD^*nD/nD$	$nD^*nD/nD^*nD^*nD^*nD+-nD$

Continued on next page

表 A.2 continued from previous page

Symbols	
$nD/nD*nD*nD+-nD*nD+-nD$	$nD/nD*nD*nD+-nD*nD*nD$
$nD+-nD*nD+-nD/nD/nD*nD$	$nD/nD+-nD+-nD/nD/nD+-nD$
$nD*nD*nD+-nD/nD+-nD+-nD$	$nD*nD+-nD/nD/nD*nD*nD$
$nD/nD+-nD/(nD+-nD+-nD)+-nD$	$nD+-nD+-nD/(nD+-nD/nD)+-nD$
$nD/(nD/nD*nD)*nD+-nD*nD$	$nD/nD*nD+-nD*(nD+-nD)*nD$
$(nD*nD*nD+-nD)*nD/nD+-nD$	$nD*nD+-nD+-nD/nD+-nD/nD$
$nD/nD+-nD+-nD*nD+-nD/nD$	$nD+-nD/nD+-nD/nD+-nD*nD$
$nD+-nD/nD+-nD/nD+-nD/nD$	$nD*nD+-nD*nD*nD+-nD/nD$
$nD*nD*nD/nD+-nD+-(nD*nD)$	$nD+-nD+-(nD/nD+-nD*nD)*nD$
$nD+-nD/nD+-nD+-nD+-nD*nD$	$nD+-nD/(nD+-nD)/nD+-nD*nD$
$nD*nD+-nD+-nD*nD+-nD/nD$	$nD+-nD/nD*(nD+-nD+-nD+-nD)$
$(nD+-nD+-nD+-nD+-nD)*nD+-nD$	$nD+-nD*nD+-nD+-nD/nD+-nD$
$nD+-nD/nD*nD+-nD*nD+-nD$	$nD+-nD+-nD*nD/nD+-nD+-nD$
$nD+-nD+-nD+-(nD/nD)+-nD/nD$	$nD/nD*nD/nD/(nD*nD)*nD$
$nD+-nD/nD*nD+-nD+-nD+-nD$	$nD*nD*nD/nD+-nD+-nD/nD$
$nD/(nD+-nD/nD)/nD/nD+-nD$	$nD/nD*nD+-nD+-nD+-nD*nD$
$nD+-nD/nD*(nD/nD)+-nD*nD$	$nD*nD*nD+-nD+-nD/nD+-nD$
$nD+-nD+-nD+-nD*nD/nD+-nD$	$nD*nD*(nD+-nD/nD*nD*nD)$
$nD*nD*nD*nD+-nD/nD+-nD$	

參考文獻

- [1] C. Humphreys and R. Parker, *Making Number Talks Matter: Developing Mathematical Practices and Deepening Understanding, Grades 4-10*. Stenhouse Publishers, 2015.
- [2] J. Boaler, *Mathematical Mindsets: Unleashing Students' Potential Through Creative Math, Inspiring Messages and Innovative Teaching*. John Wiley & Sons, 2015.
- [3] T. Liu and B. K. H. Low, “Goat: Fine-tuned llama outperforms gpt-4 on arithmetic tasks,” *arXiv preprint arXiv:2305.14201*, 2023.
- [4] Y. Zhen, D. Ming, L. Qingsong, J. Zhihuan, H. Zehai, G. Yuyi, B. Jinfeng, and T. Jie, “Gpt can solve mathematical problems without a calculator,” *arXiv preprint arXiv:2309.03241*, 2023.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [6] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le, “Xlnet: Generalized autoregressive pretraining for language understanding,” in *Advances in neural information processing systems*, vol. 32, 2019.
- [7] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li,

- and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [8] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving language understanding by generative pre-training,” 2018. OpenAI Blog.
- [9] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Łukasz Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in neural information processing systems*, vol. 30, 2017.
- [11] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [12] A. Mitra, H. Khanpour, C. Rosset, and A. Awadallah, “Orca-math: Unlocking the potential of slms in grade school math,” *arXiv preprint arXiv:2402.14830*, 2024.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
- [14] S. Jelassi, S. d’ Ascoli, C. Domingo-Enrich, Y. Wu, Y. Li, and F. Charton, “Length generalization in arithmetic transformers,” *arXiv preprint arXiv:2306.15400*, 2023.

自傳與學術成就

我是李昌育，出生於台灣，於 2022 年畢業於國立臺灣師範大學電機工程學系。出於對於人工智慧與教育領域的研究興趣，進入現指導教授實驗室，致力於開發教育科技的應用，並選擇以多樣化解題系統作為研究題目，以不斷提升自我能力。

在大學期間，以題目「深度知識追蹤」獲得學系畢業專題展的佳作獎項。於 2024 年的 ICCE TW 研討會以第一作者的身分發表題目為 Enhancing Solution Diversity in Arithmetic Problems using Fine-Tuned AI Language Model 的論文。

